

Distributed Optimization for Machine Learning

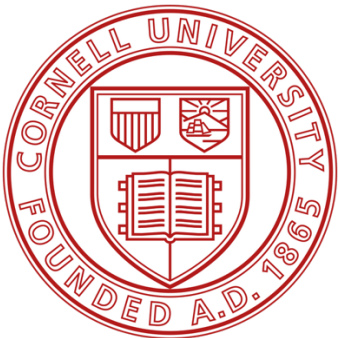
Lecture 22 – Analog computing for energy-efficient AI: Part II

ECE 5290/7290 & ORIE 5290

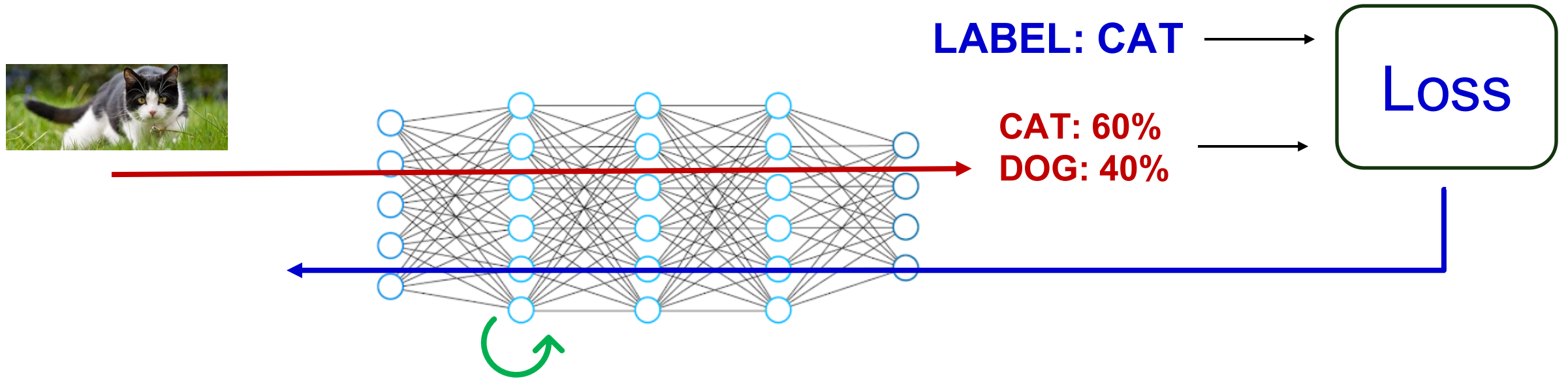
Tianyi Chen

School of Electrical and Computer Engineering
Cornell Tech, Cornell University

November 19, 2025



Standard model training framework



Model training involves three phases:

1. **Forward**: Data propagates through multiple layers, yielding a **Loss**
2. **Backward**: Take the gradient of Loss w.r.t. weights
3. **Weight update**: Update weights by gradient-based methods (SGD, Adam, etc)

Review: back-propagation of linear layer

Consider the BP process in fully connected layers
(w.o. any activation function / bias term)

Objective $f(W)$

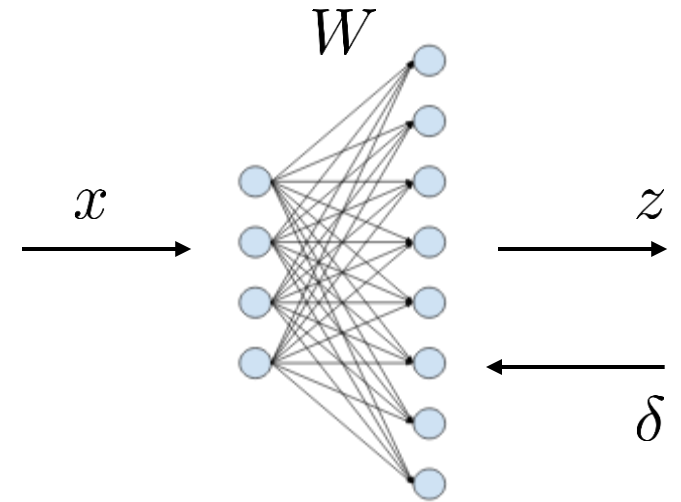
Backward signal $\delta := \frac{\partial f}{\partial z}$

Backward signal for next layer

$$\delta' := \frac{\partial f}{\partial x} = \frac{\partial f}{\partial z} \frac{\partial z}{\partial x} = \delta W \quad (\text{MVM})$$

Backward signal for next layer

$$\frac{\partial f}{\partial W} = \frac{\partial f}{\partial z} \frac{\partial z}{\partial W} = \delta \otimes x \quad (\text{output product of two vectors})$$



Review: Gradient-based update

Stochastic Gradient Descent (SGD) on W_k

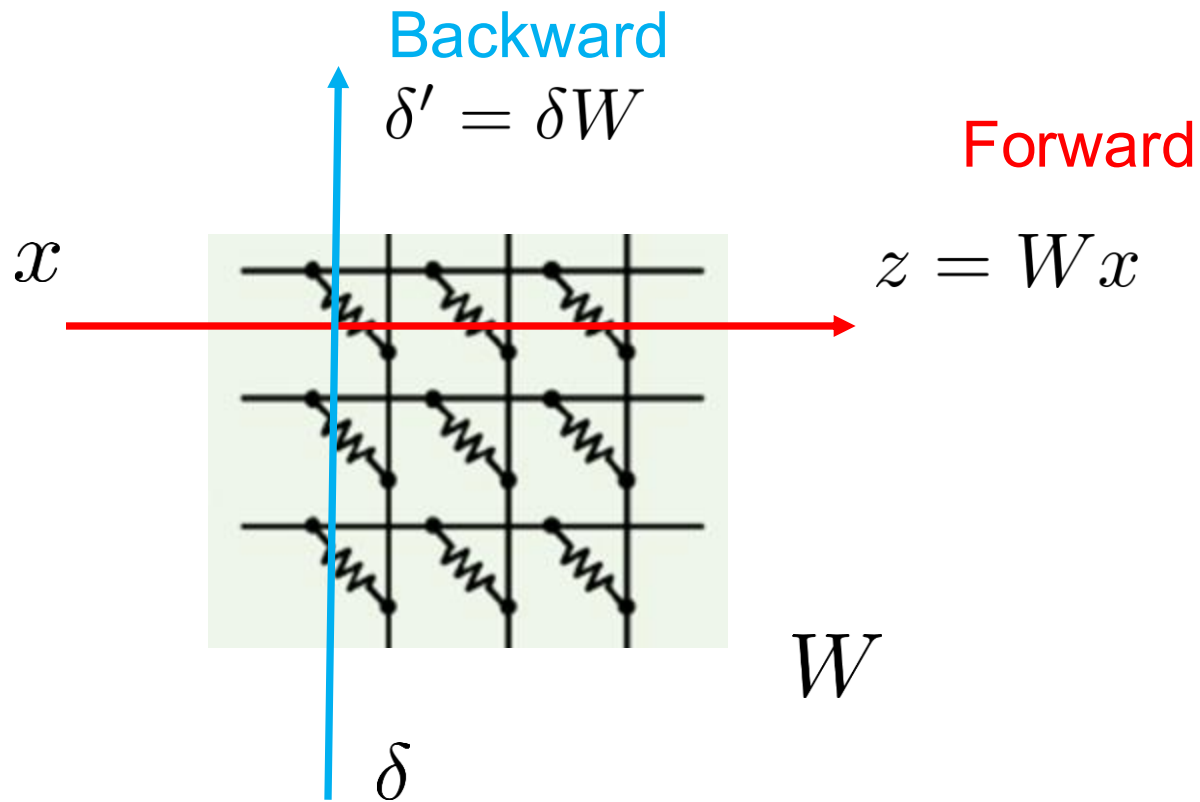
$$W_{k+1} = W_k - \underset{\substack{\uparrow \\ \text{step size}}}{\alpha} \frac{\partial f}{\partial W} = W_k - \alpha \underset{\substack{\uparrow \\ \text{Rank-1 matrix}}}{\delta \otimes x}$$

AIMC hardware leverages the rank-1 structure of the gradient for acceleration

$$w_{k+1}^{ij} = w_k^{ij} - \alpha \delta_i x_j$$

MVM in forward and backward passes

Consider a fully connected layer (w.o. activation function / bias term)



Back-propagation

$$\delta := \frac{\partial f}{\partial z} \quad \text{Gradient of output}$$

$$\begin{aligned} \delta' &:= \frac{\partial f}{\partial x} && \text{Gradient of input} \\ &= \frac{\partial f}{\partial z} \frac{\partial z}{\partial x} \\ &= \delta W \end{aligned}$$

Forward and backward relies on analog MVM operator

Analog stochastic gradient descent

Gradient descent by stochastic computing

$$w_{k+1}^{ij} = w_k^{ij} - \alpha \delta_i x_j$$

Send pulses on row/column randomly

$$p(\text{send pulse on } i\text{-th row}) = C x_i$$

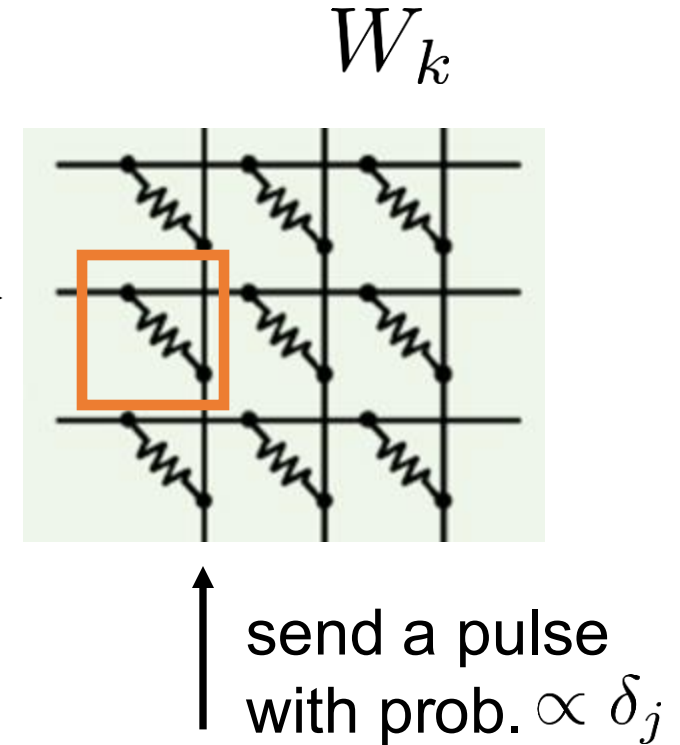
$$p(\text{send pulse on } j\text{-th column}) = C \delta_j$$

Probability two pulses coincide in (i, j)-th element

$$p(w^{ij} \text{ is updated}) = C^2 x_i \delta_j$$

Expected update after B pulse $\mathbb{E}[w_{k+1}^{ij}] = w_k^{ij} + B \cdot \Delta w_{\min} \cdot C^2 \delta_i x_j$

send a pulse
with prob. $\propto x_i$



Analog stochastic gradient descent (con't)

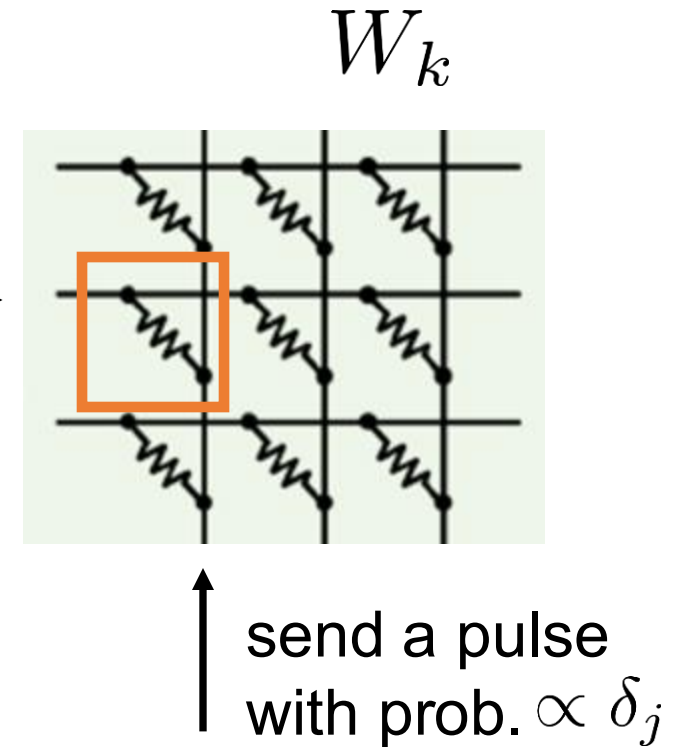
Expected update after B pulse

$$\mathbb{E}[w_{k+1}^{ij}] = w_k^{ij} + B \cdot \Delta w_{\min} \cdot C^2 \delta_i x_j$$

Properly selection of B recovers SGD

$$w_{k+1}^{ij} = w_k^{ij} - \alpha \delta_i x_j$$

send a pulse
with prob. $\propto x_i$

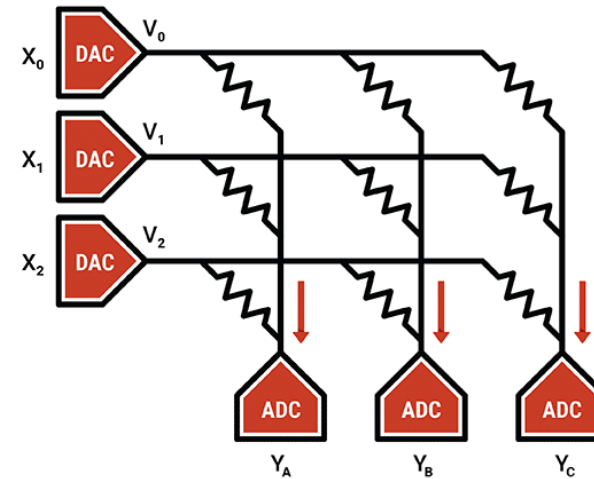
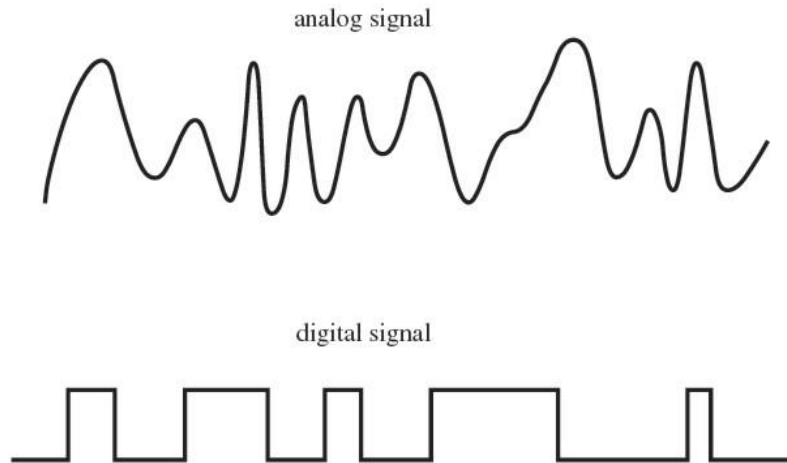


	Digital	Analog
MAC	$O(N^2)$	$O(1)$

Analog SGD updates all elements parallelly

Outline

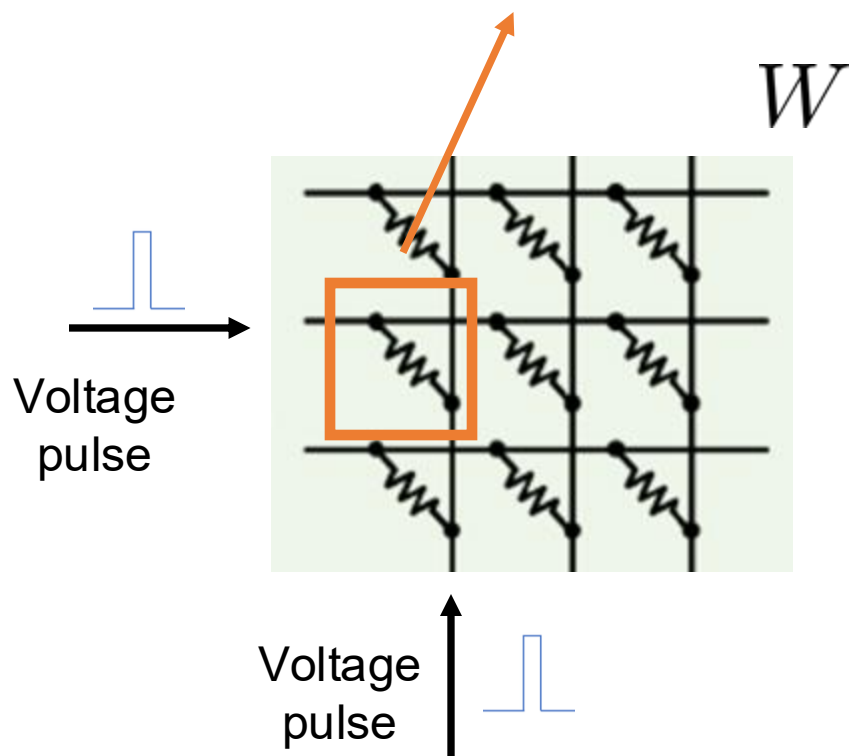
- Challenges in Analog Training
- From Linear to General Devices
- Advanced Topics in Analog Training



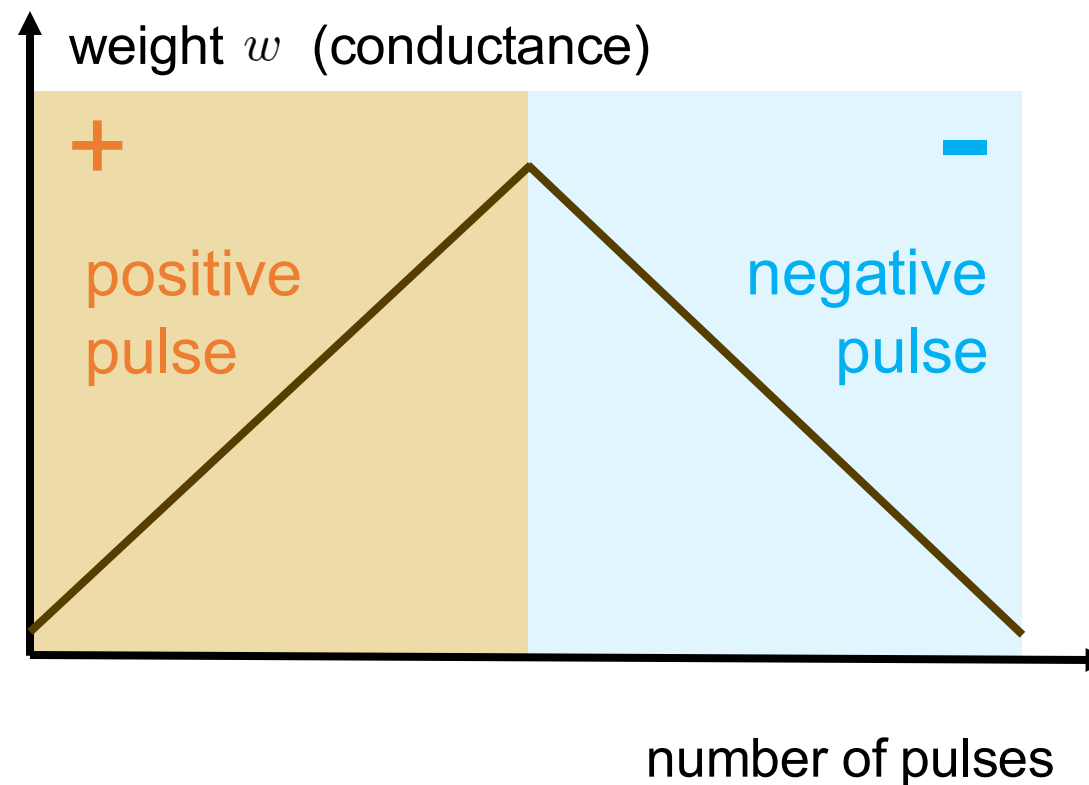
Hardware imperfection: asymmetric update

Analog SGD is efficient. However, it suffers from **asymmetric weight update**

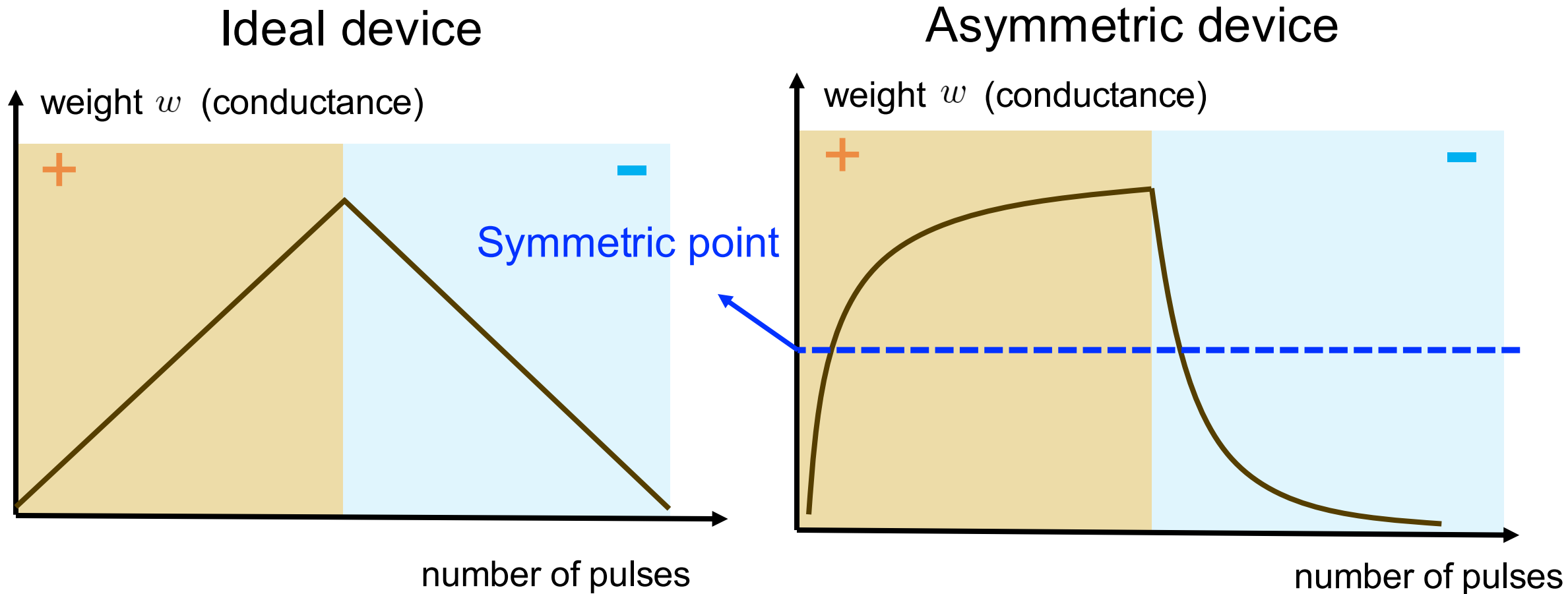
Conductance increase/decrease
when pulses coincide



Ideal symmetric device



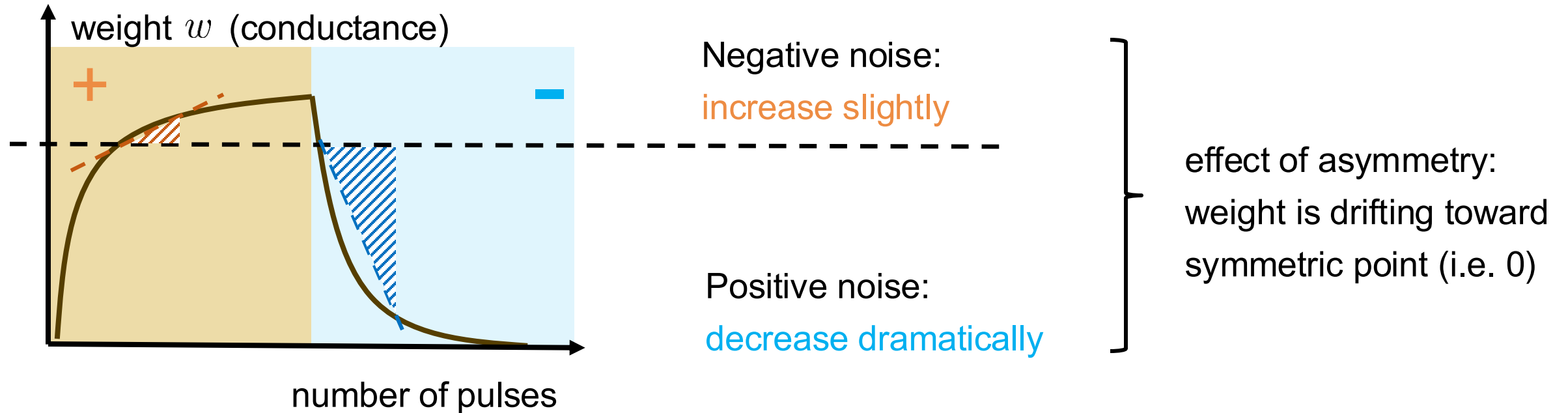
Hardware imperfection: asymmetric update



Response of RPU varies by direction

Asymmetric update leads to instability training

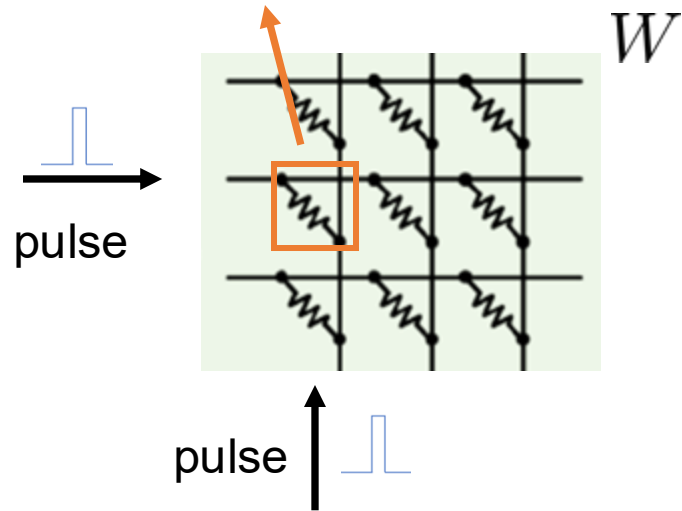
The gradient is usually noisy $\text{Var}[f(W)] \neq 0$



Gradient noise pushes the weight to symmetric point

Zero-shifting

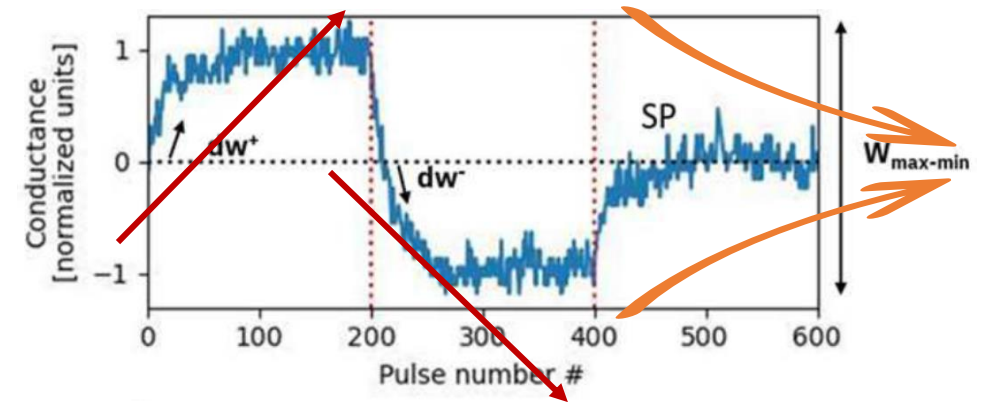
Update weight (conductance) by pulse update



$\min \text{Loss}(\mathbf{W})$
over Model weight ($\mathbf{W}$)

“Ideal” loss

Conductance increases/decreases follow different trajectory



Attracted by a “symmetric point”

$\min \text{Loss}(\mathbf{W}) + \sigma \|\mathbf{W} - \text{SP}\|^2$
over Model Weight ($\mathbf{W}$)

“Ideal” loss

Challenge: Limited number of states

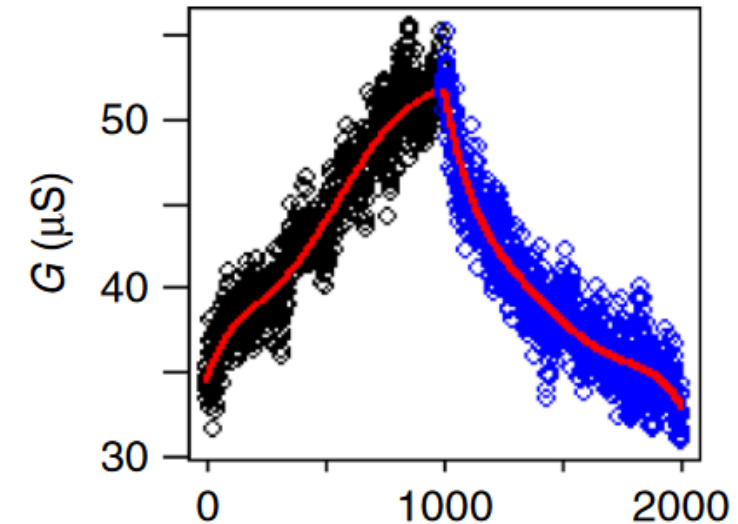
Pulse update: Send pulses to change conductance (thus weight)

Update granularity: $\Delta w_{\min}$ (Controlled by pulse shape)

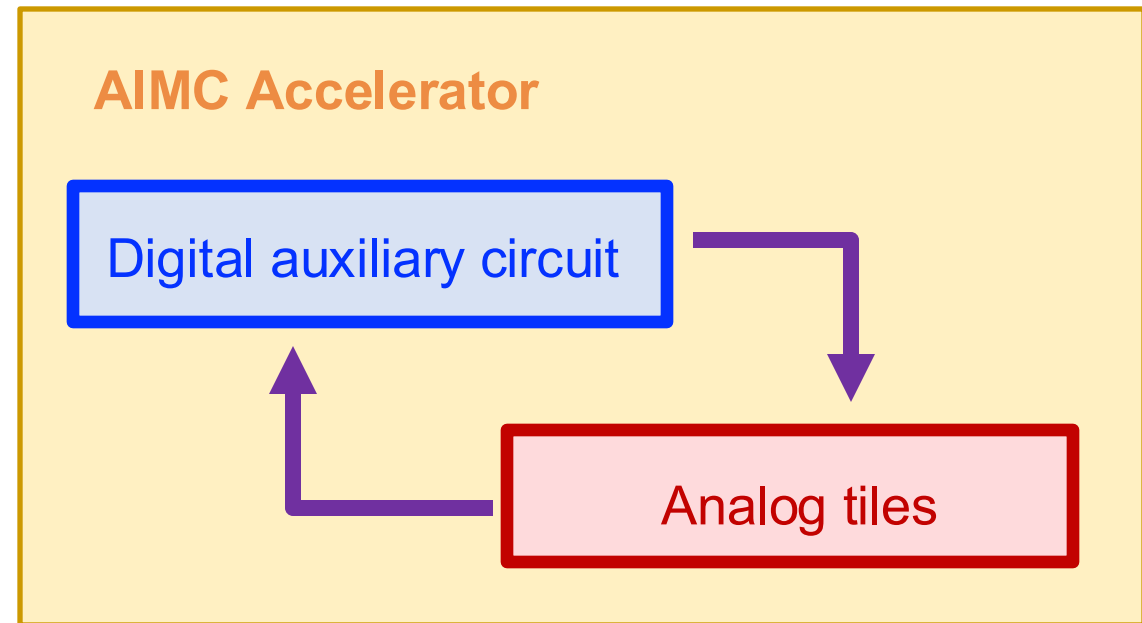
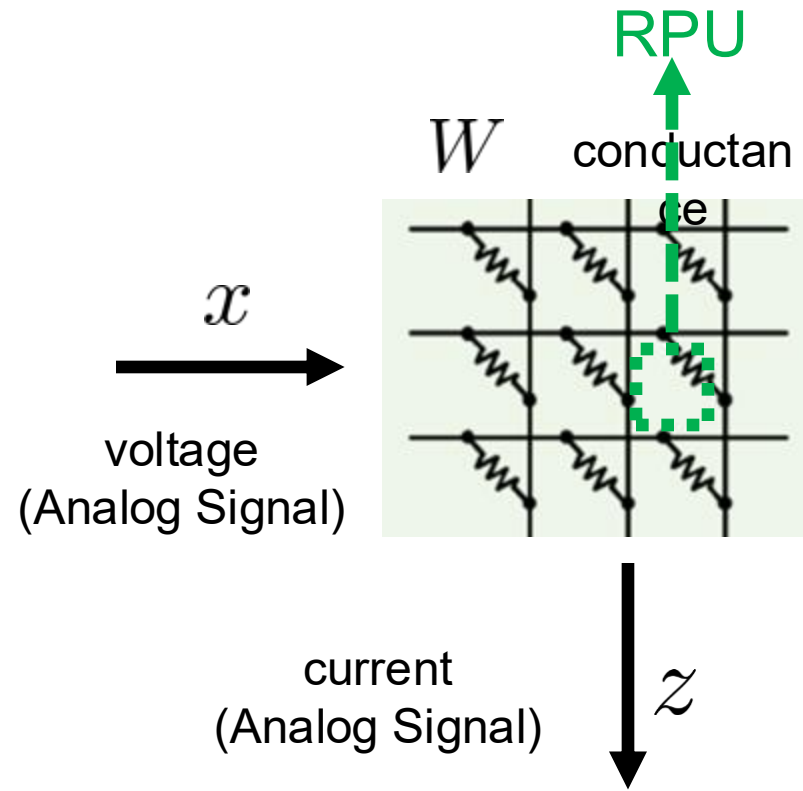
Number of states:

(max weight – min weight) / $\Delta w_{\min}$

Typically 10-1000

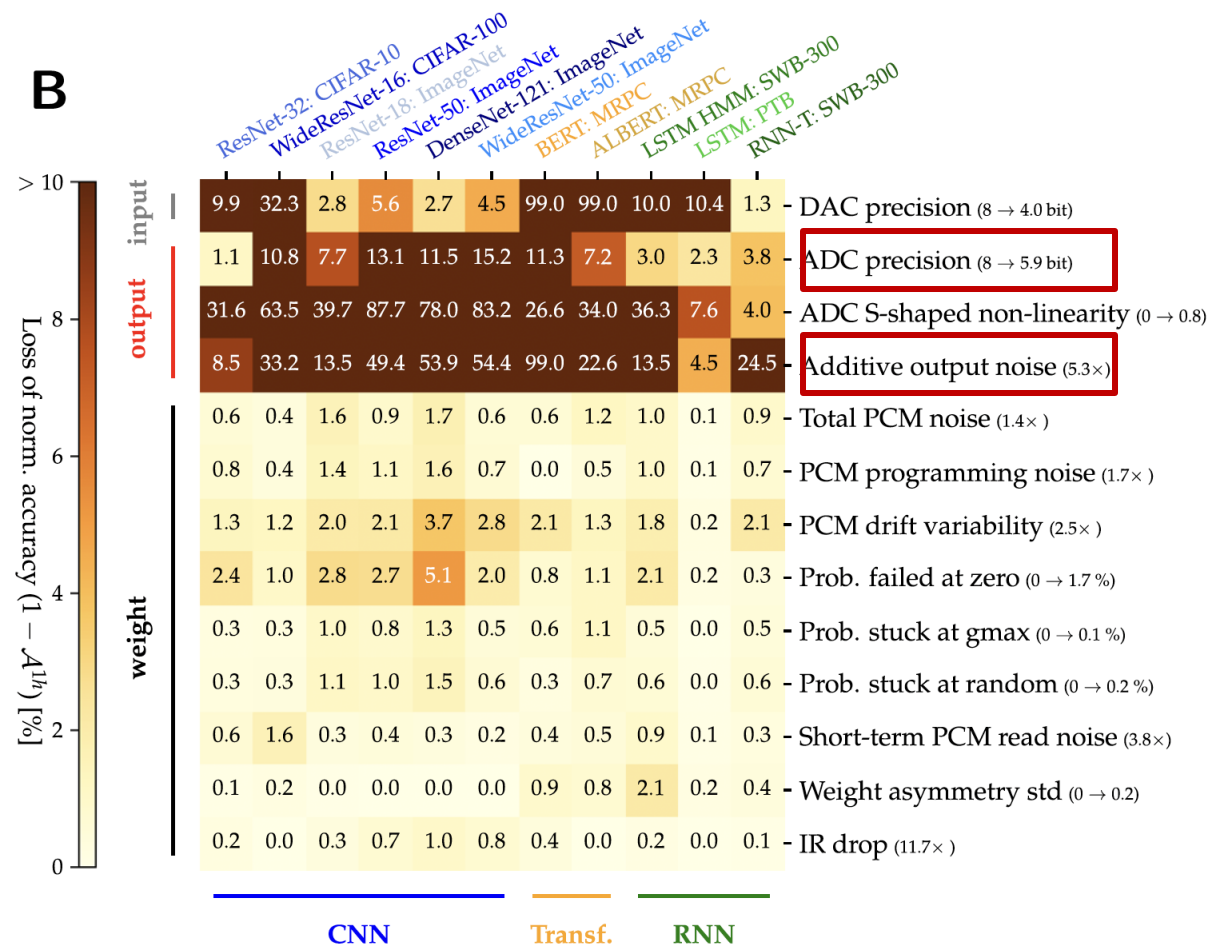


Other non-idealities: reading noise and ADC noise



Analog device has reading noise: additive noise and analog-digital converter error

Other non-idealities: reading noise and ADC noise



Analog device has reading noise: additive noise and analog-digital converter error

How to alleviate the asymmetric issue?

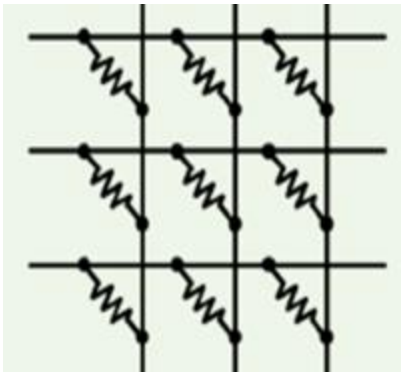
Reduce the update noise?

Analog SGD -> Analog “Batch” SGD

- Gradient computed on one sample is always noisy
- How about “batch” SGD?

Idea: introduce a “gradient tile” to approximate “batch” gradient

gradient tile P_k

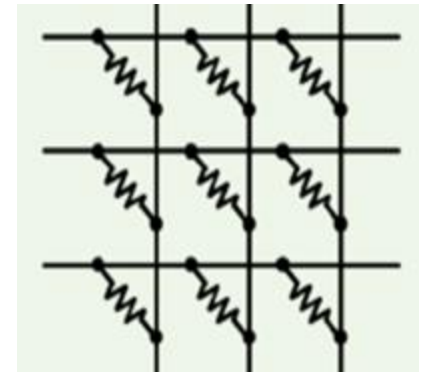


$\approx$

$$\nabla f(W_k)$$

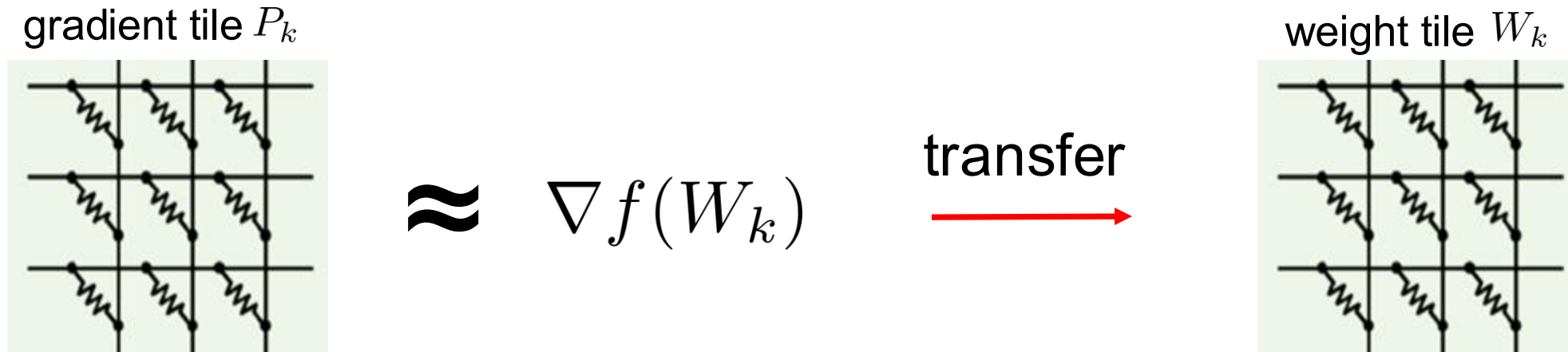
transfer


weight tile W_k



Implementation of Analog “Batch” SGD

- Gradient accumulation is implemented by pulse update
- Transfer is implemented by “weight copy” operation
- Use “parallel update” to paste the update on weight tile



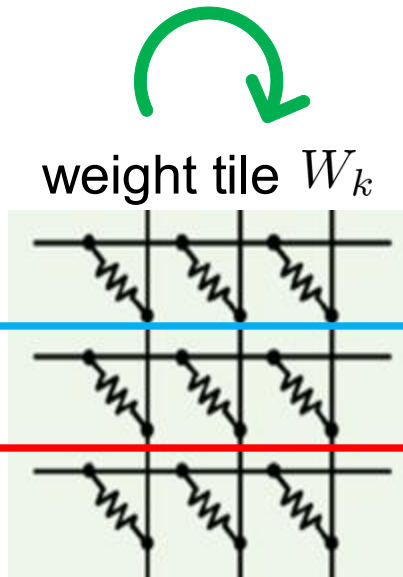
Transfer one column at each time

Comparing with Analog SGD

We reach the heuristic Tiki-Taka algorithm

(Gokmen & Wilfried, 2020)

3. Noisy Gradient



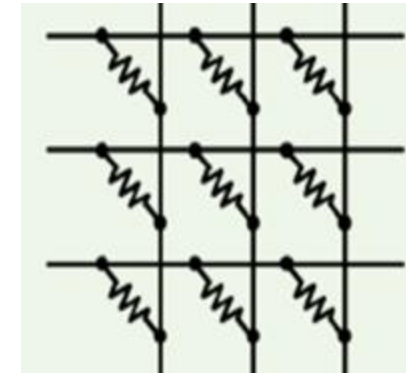
Analog SGD

3. Noisy Gradient

2. Backward

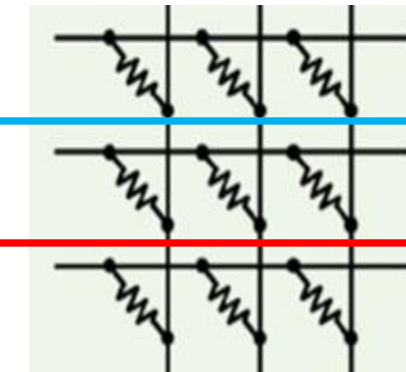
1. Forward

gradient tile P_k



4. Transfer

weight tile W_k



Tiki-Taka

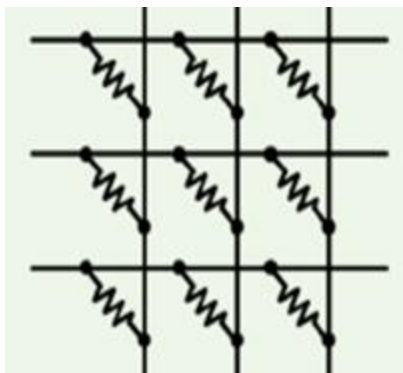
How does Tiki-Taka work?

Tiki-Taka works well in practice

But why does it work?

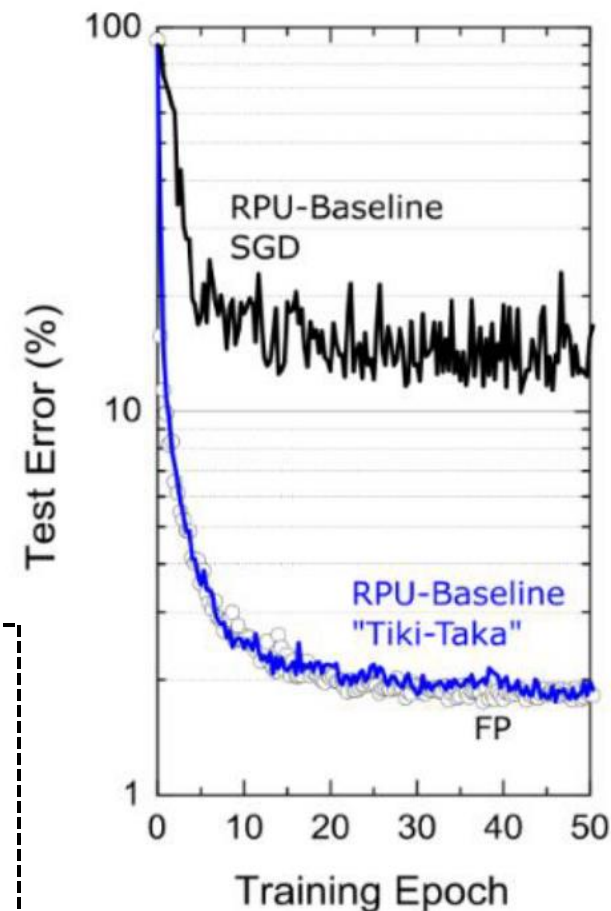
Given the update is asymmetric, how can we ensure gradient tile is a “batch” gradient?

gradient tile P_k



$$\approx \nabla f(W_k)$$

???



How to mathematically model this issue?

How to model analog SGD? Analog SGD dynamic

Rewrite the update in a compact form?

Symmetric decomposition

$$W_{k+1} = U(W_k, \Delta W) \quad U(w, \Delta w) := \begin{cases} w + \Delta w \cdot q_+(w) , & \Delta w \geq 0, \\ w + \Delta w \cdot q_-(w) , & \Delta w < 0, \end{cases}$$
$$= w + \Delta w \odot F(w) - |\Delta w| \odot G(w)$$

where $F(w) := \frac{1}{2}(q_-(w) + q_+(w))$, $G(w) := \frac{1}{2}(q_-(w) - q_+(w))$

Symmetric part

Asymmetric part

$$W_{k+1} = W_k + \Delta W \odot F(W_k) - |\Delta W| \odot G(W_k)$$

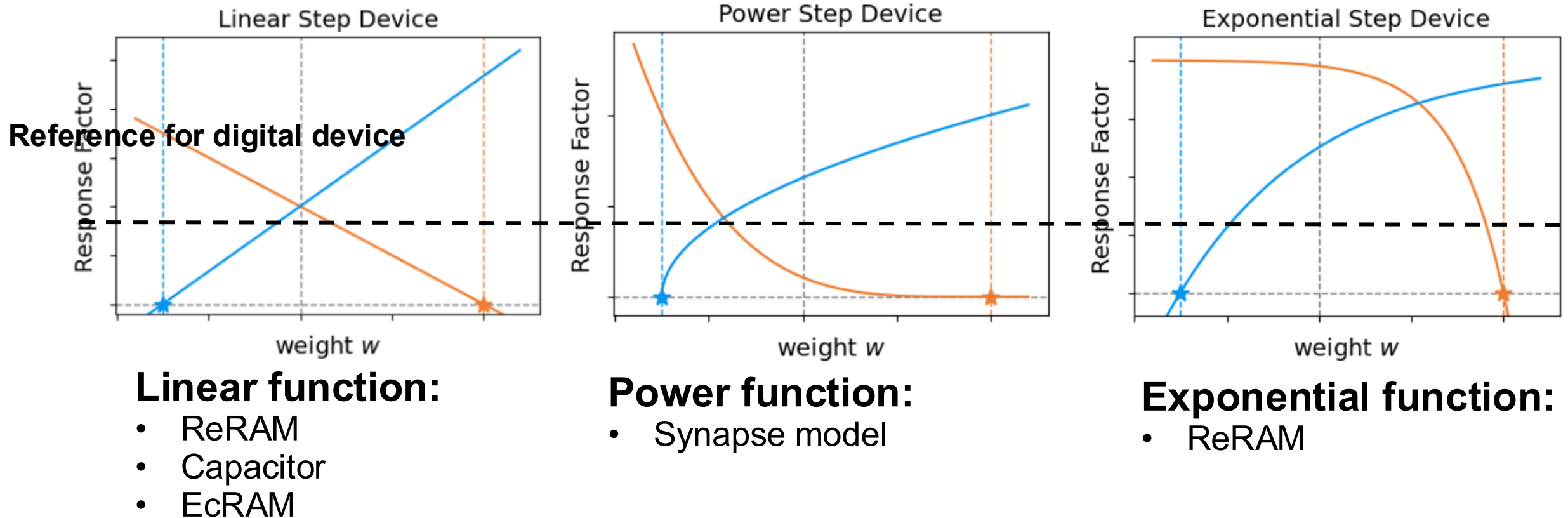
$$-\alpha(\nabla f(W_k) + \varepsilon_k)$$

Unified form for generic devices

Different devices have different response factors

Response factor: the rate of conductance change per pulse

— $q_+(w)$
— $q_-(w)$



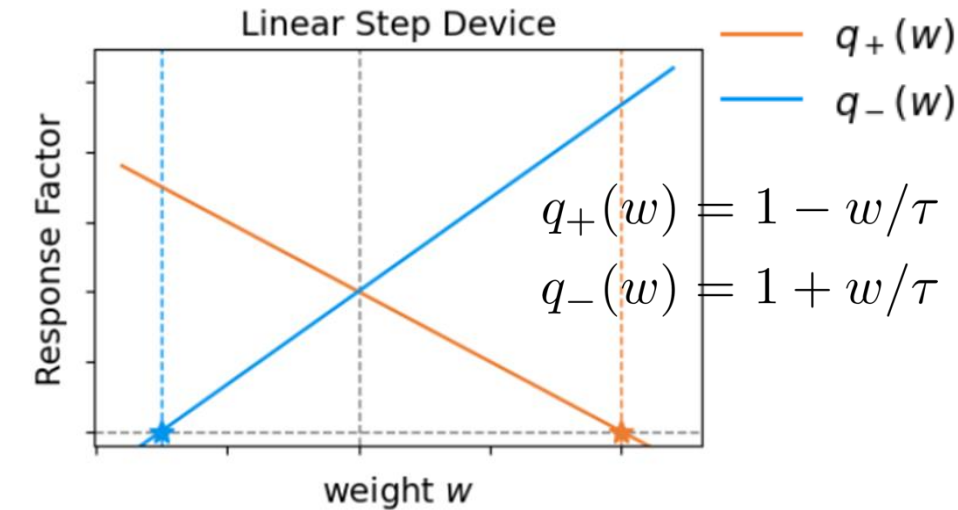
Special case: Analog SGD on asymmetric linear device

We begin from the **linear devices**, i.e. both $q_+(\cdot)$, $q_-(\cdot)$ are linear functions

$$W_{k+1} = W_k + \Delta W - \frac{1}{\tau} |\Delta W| \odot W_k$$

■

$$-\alpha(\nabla f(W_k) + \varepsilon_k)$$



Cover a series of materials: ReRAM, Capacitor, EcRAM

Simple yet representative enough to capture the non-ideality

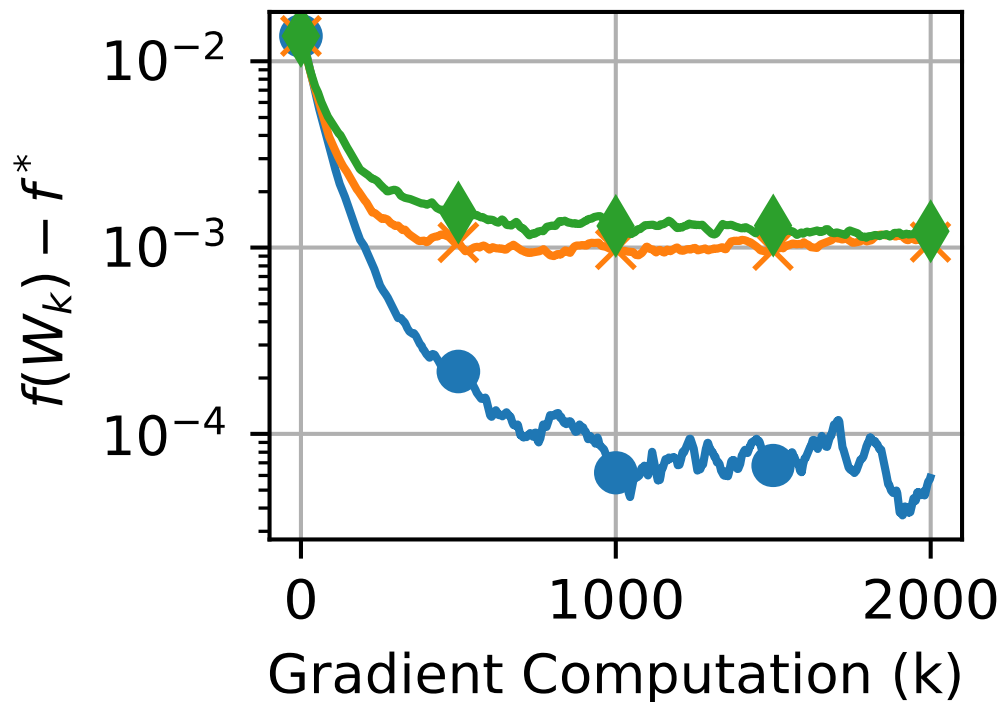
T Stecconi, et al. "Analog Resistive Switching Devices for Training Deep Neural Networks with the Novel Tiki-Taka Algorithm." *Nano Letters* **24.3** (2024): 866-872.

J Won, et al. "Device-Algorithm Co-Optimization for an On-Chip Trainable Capacitor-Based Synaptic Device with IGZO TFT and Retention-Centric Tiki-Taka Algorithm."

Advanced Science **10.29** (2023): 2303018.

J Tang, et al. "ECRAM as scalable synaptic cell for high-speed, low-power neuromorphic computing." *IEDM*, 2018.

Verification of the proposed analog SGD dynamic



$$W_{k+1} = W_k - \alpha(\nabla f(W_k) + \varepsilon_k)$$

VS

$$W_{k+1} = W_k - \alpha(\nabla f(W_k) + \varepsilon_k) - \frac{\alpha}{\tau} |\nabla f(W_k) + \varepsilon_k| \odot W_k$$

—●— Digital SGD Dynamic —×— Analog SGD Dynamic —◆— Analog SGD Simulation

The proposed dynamic matches the analog training better!

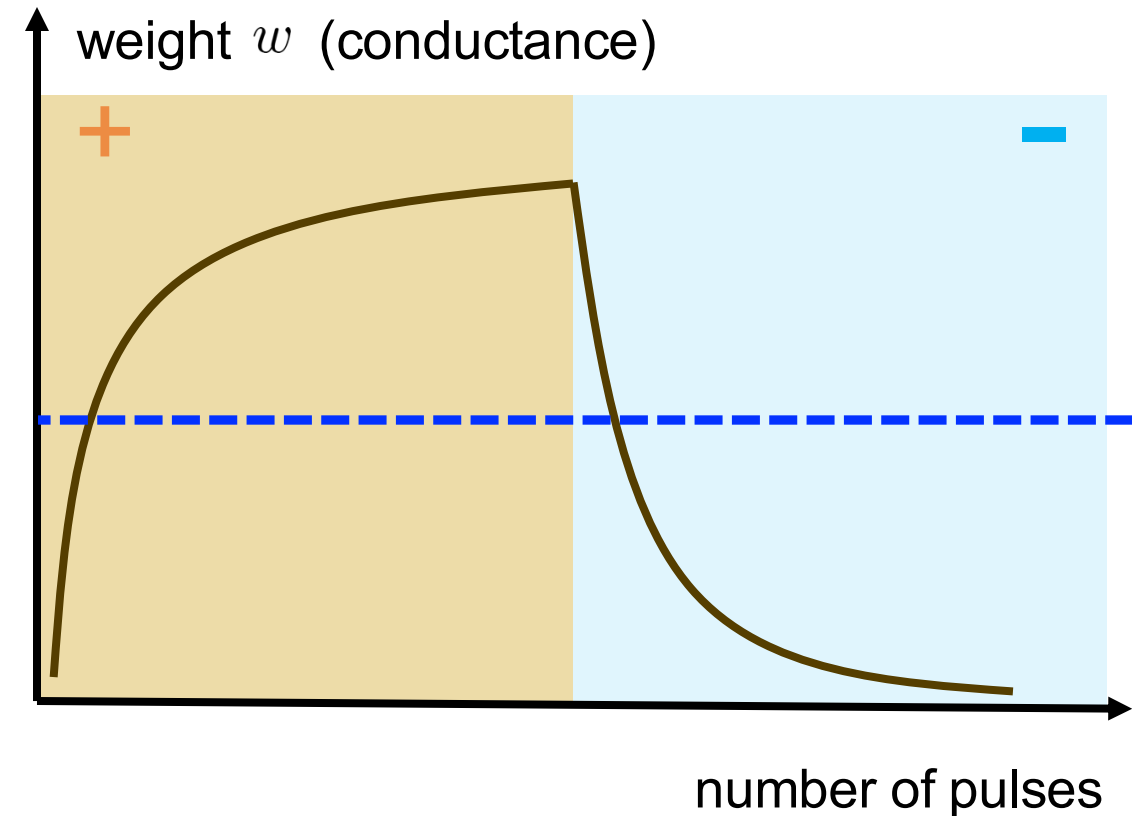
Physics property 1: Attraction at symmetric point

$$W_{k+1} = W_k + \Delta W - \frac{1}{\tau} |\Delta W| \odot W_k$$

the asymmetric component is 0 if $W_k = 0$

$q_+(W_k) = q_-(W_k)$ at the **symmetric point (SP)**

Asymmetric Device



Analog SGD behaves like Digital SGD near the symmetric point

Physics property 1: Attraction at symmetric point

$$W_{k+1} = W_k + \Delta W - \frac{1}{\tau} |\Delta W| \odot W_k$$

Moving away: If $\text{sign}(\Delta W) = \text{sign}(W_k)$

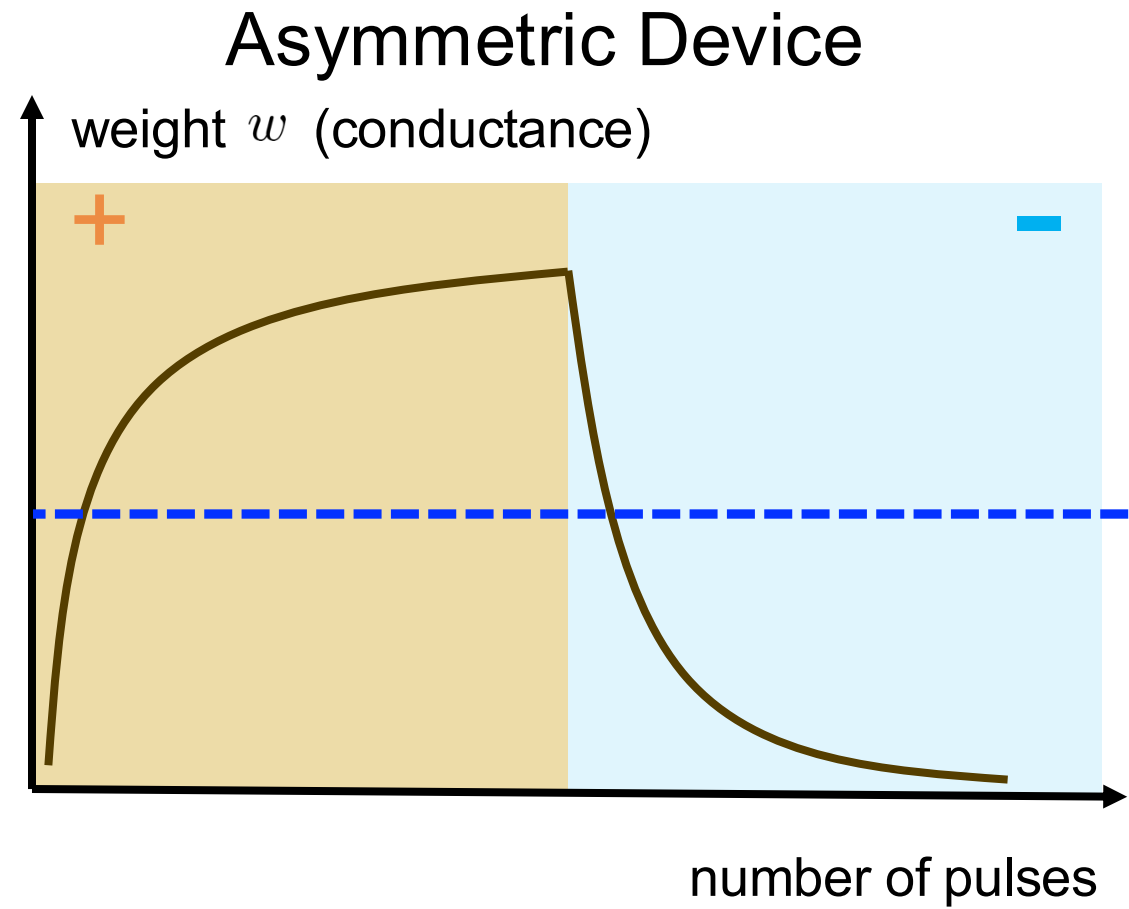
$$|W_{k+1} - W_k| = |1 - |W_k|/\tau| \cdot |\Delta W|$$

Moving away from symmetric point: slow

Moving back: If $\text{sign}(\Delta W) = -\text{sign}(W_k)$

$$|W_{k+1} - W_k| = |1 + |W_k|/\tau| \cdot |\Delta W|$$

Moving to symmetric point: fast



Physics property 2: Bounded weight

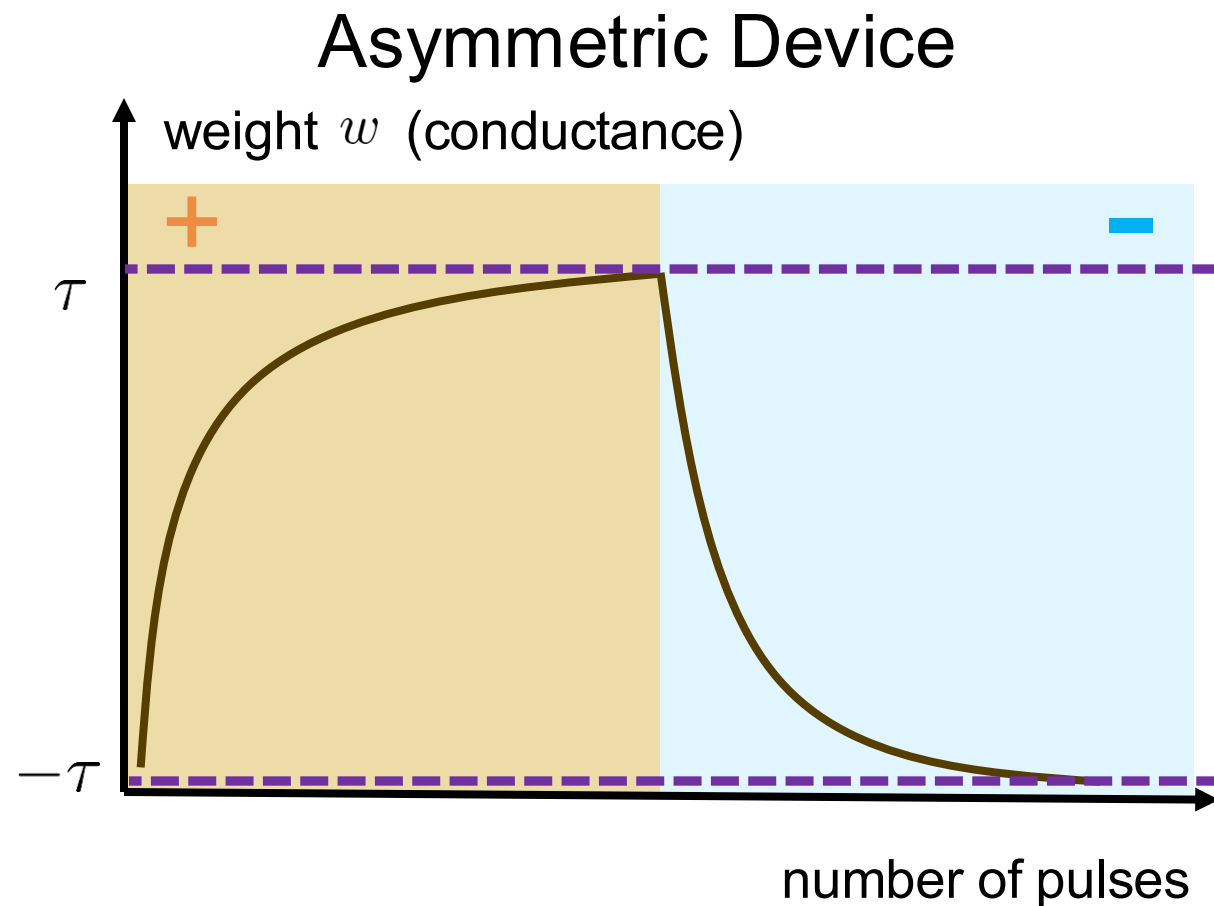
$$W_{k+1} = W_k + \Delta W - \frac{1}{\tau} |\Delta W| \odot W_k$$

A natural conclusion of saturation: bounded weight

Theorem 1 (Bounded)

If the increment $|\Delta w| \leq \tau$, it holds that

$$\|W_k\|_{\infty} \leq \tau, \forall k \in \mathbb{N}.$$



The weight is the bounded inside an “dynamic range”

Implicit bias to low-complexity model!

How do **these intricate properties** affect the training error?

Convergence upper bound of Analog SGD

Assumption 1 (Bounded variance) $\mathbb{E}[\varepsilon_k] = 0, \mathbb{E}[\|\varepsilon_k\|^2] \leq \sigma^2$

Theorem 2 (Convergence)

Under the above assumption, if the step size is set as $\alpha = O(1/\sqrt{\sigma^2 K})$ and K is sufficiently large such that, Analog SGD satisfies

$$\frac{1}{K} \sum_{k=0}^{K-1} \mathbf{E}[\|\nabla f(W_k)\|^2] \leq \mathcal{O} \left(\frac{1}{1 - W_{\max}^2/\tau^2} \sqrt{\frac{\sigma^2}{K}} \right) + \boxed{\frac{4\sigma^2 W_{\max}^2/\tau^2}{1 - W_{\max}^2/\tau^2}}.$$

where τ measures the asymmetry and degree of saturation.

asymptotic error

Analog SGD has asymptotic error, thus not convergent!

Lower bound (a.k.a. Limit) of asymptotic error

Theorem 3 (Lower Bound)

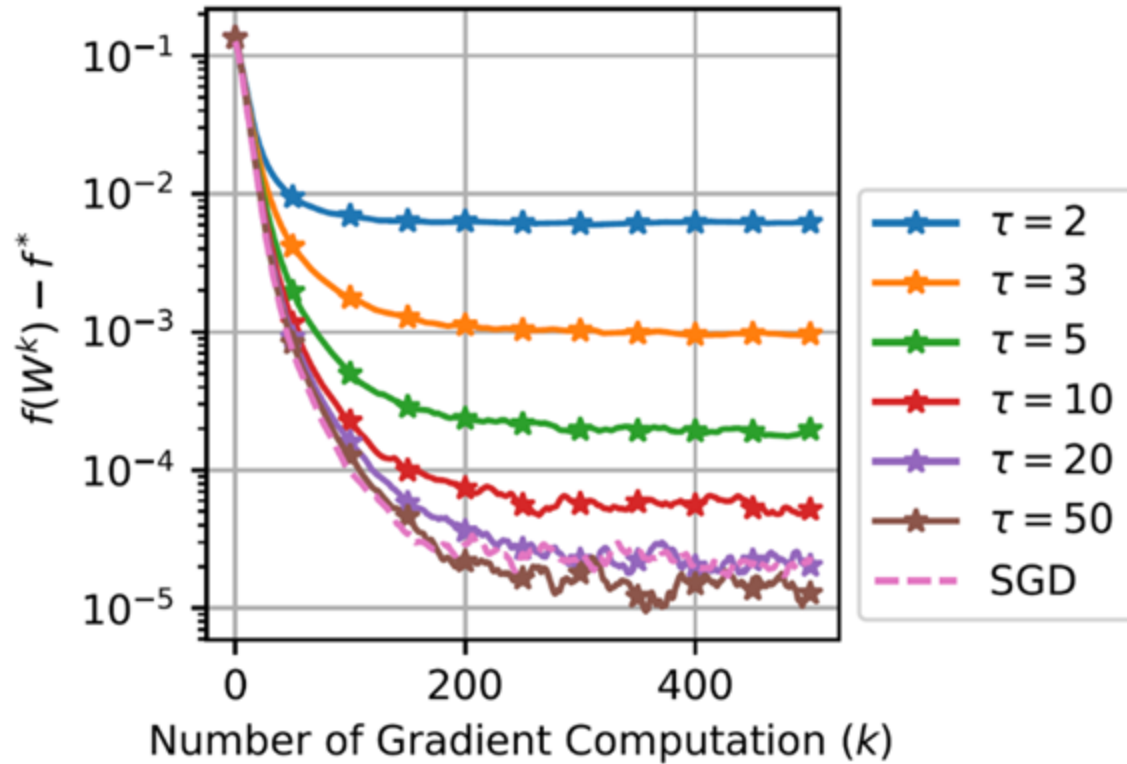
There is an instance satisfying the above assumptions, such that Analog SGD generates the sequence that satisfies

$$\frac{1}{K} \sum_{k=0}^{K-1} \mathbf{E}[\|\nabla f(W_k)\|^2] = \Omega \left(\frac{4\sigma^2 W_{\max}^2 / \tau^2}{1 - W_{\max}^2 / \tau^2} + \sqrt{\frac{\sigma^2}{K}} \right).$$

The asymptotic error is not an artifact of the analysis!

Ablation study: Asymptotic error of Analog SGD

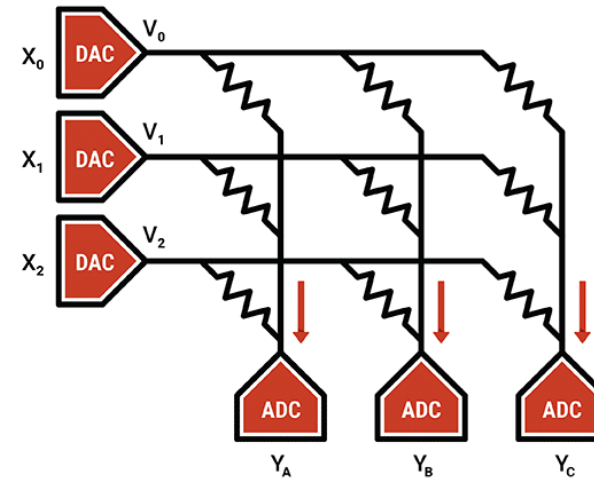
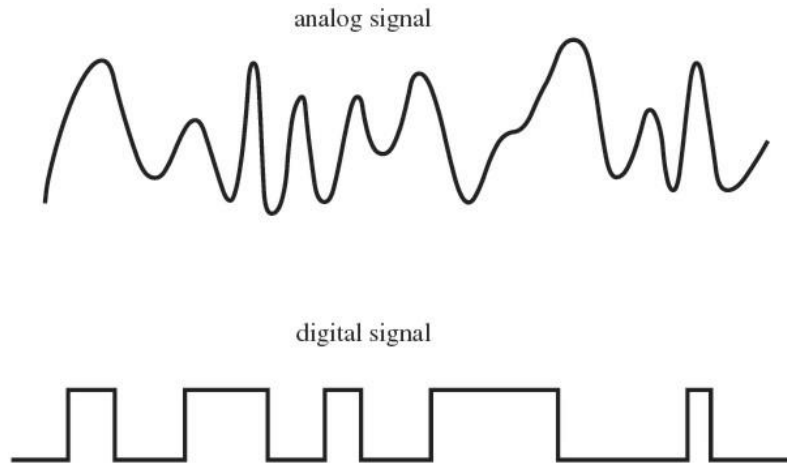
Saturation range



Algorithms to overcome the hardware non-ideality?

Outline

- Challenges in Analog Training
- From Linear to General Devices
- Advanced Topics in Analog Training



How to eliminate the asymptotic error?

Intuition: asymptotic error is proportional to the noise variance

$$\frac{1}{K} \sum_{k=0}^{K-1} \mathbf{E}[\|\nabla f(W_k)\|^2] = \Omega \left(\boxed{\frac{4\sigma^2 W_{\max}^2 / \tau^2}{1 - W_{\max}^2 / \tau^2}} + \frac{1}{\sqrt{K}} \right).$$


asymptotic error

Asymmetric update + update noise $\Rightarrow$ weight drift $\Rightarrow$ asymptotic error

Reduce update noise $\Rightarrow$ Eliminate asymptotic error?

How to eliminate the asymptotic error?

$$W_{k+1} = W_k - \alpha(\nabla f(W_k) + \varepsilon_k) - \frac{\alpha}{\tau} |\nabla f(W_k) + \varepsilon_k| \odot W_k$$

Observation: asymptotic error comes from the **mismatching!**

$$\nabla f(W^*) = 0$$



$$W^* = 0$$

algorithmic critical point

physical stationary point

Gradient signals are always noisy

Noise attracts the weight to symmetric point

Construct a variable to make them match?

Residual variable

Solution: introduce another tile P_k to track the **residual** of W_k

$$P^*(W) \in \arg \min_{P \in \mathbb{R}^D} f(W + \gamma P)$$

and then update W_k by transferring P_k to W_k

Why it works?

$$P_k \rightarrow P^*(W^*) \rightarrow 0 \quad \text{as} \quad W_k \rightarrow W^*$$

algorithmic critical point $\iff$ physical stationary point

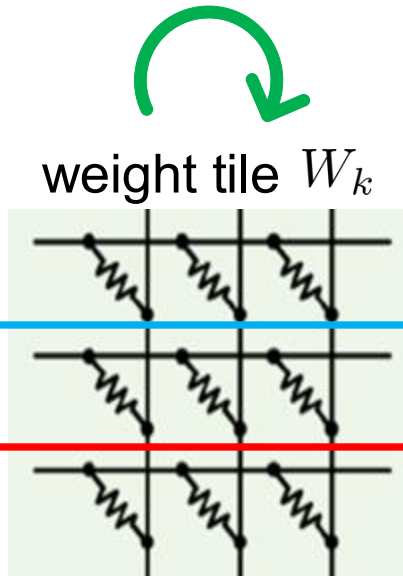
Two points matches on the residual tile

Comparing with Analog SGD

We reach the heuristic Tiki-Taka algorithm

(Gokmen & Wilfried, 2020)

3. Noisy Gradient



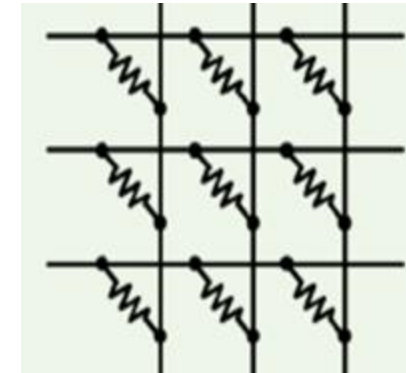
Analog SGD

3. Noisy Gradient

2. Backward

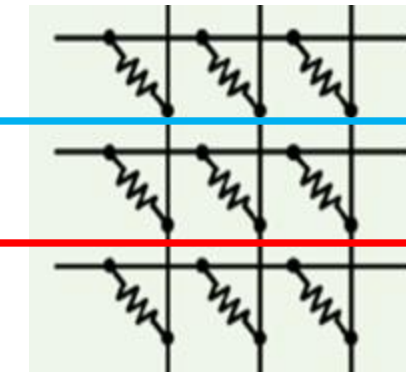
1. Forward

gradient tile P_k



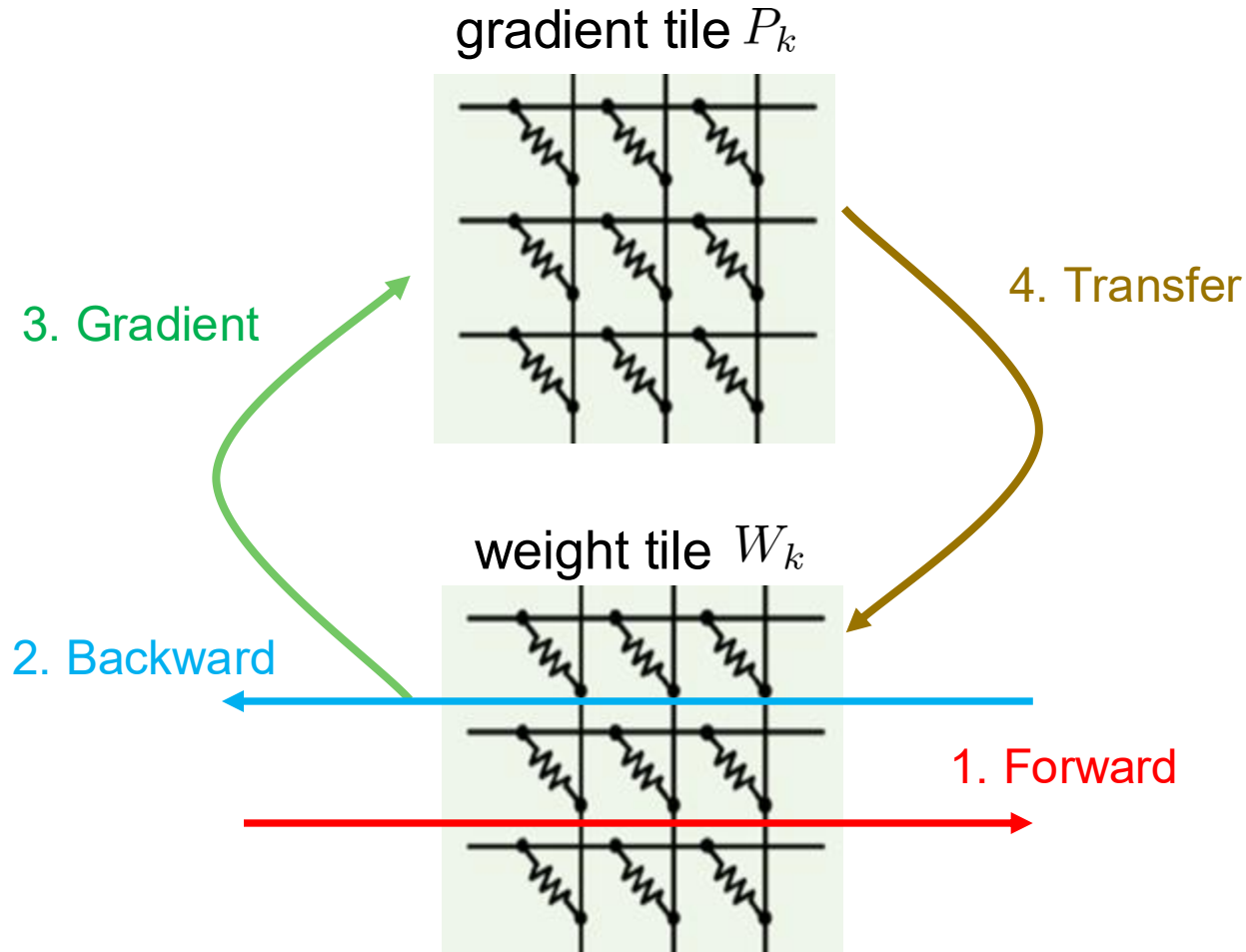
4. Transfer

weight tile W_k



Tiki-Taka

Tiki-Taka as a two-sequences update



3. Gradient (fast-time scale)

$$P_{k+1} = P_k + \beta(\nabla f(W_k + \gamma P_k) + \varepsilon_k) - \frac{\beta}{\tau} |\nabla f(W_k + \gamma P_k) + \varepsilon_k| \odot P_k$$

4. Transfer (slow-time scale)

$$W_{k+1} = W_k - \alpha P_{k+1} - \frac{\alpha}{\tau} |P_{k+1}| \odot W_k$$

Analog “momentum” reduces the weight drift

Q: Why does analog momentum work? **A:** It reduces the weight drift.

Consider the case that $\nabla f(W_k) = P_k = 0$

Analog SGD

$$\underbrace{\mathbb{E}[W_{k+1}] - W_k}_{\text{weight drift}} = -\alpha \mathbb{E}[\varepsilon_k] - \frac{\alpha}{\tau} \mathbb{E}[|\varepsilon_k|] \odot W_k = -\frac{\alpha}{\tau} \mathbb{E}[|\varepsilon_k|] \odot W_k \propto \alpha$$

weight drift

smaller weight drift



Tiki-Taka

$$\underbrace{\mathbb{E}[W_{k+1}] - W_k}_{\text{weight drift}} = -\alpha \mathbb{E}[P_{k+1}] - \frac{\alpha}{\tau} \mathbb{E}[|P_{k+1}|] \odot W_k = -\frac{\alpha\beta}{\tau} \mathbb{E}[|\varepsilon_k|] \odot W_k \propto \alpha\beta.$$

weight drift

Exact convergence of analog training

Theorem 3 (Exact convergence)

Under the proper assumptions, if the step size is set as $\alpha = O(\beta) = O(1/\sqrt{\sigma^2 K})$, the gradient is bounded, i.e. $\mathbb{E}[\|P_k\|_\infty] \leq P_{\max}$, analog momentum (aka **Tiki-Taka**) satisfies

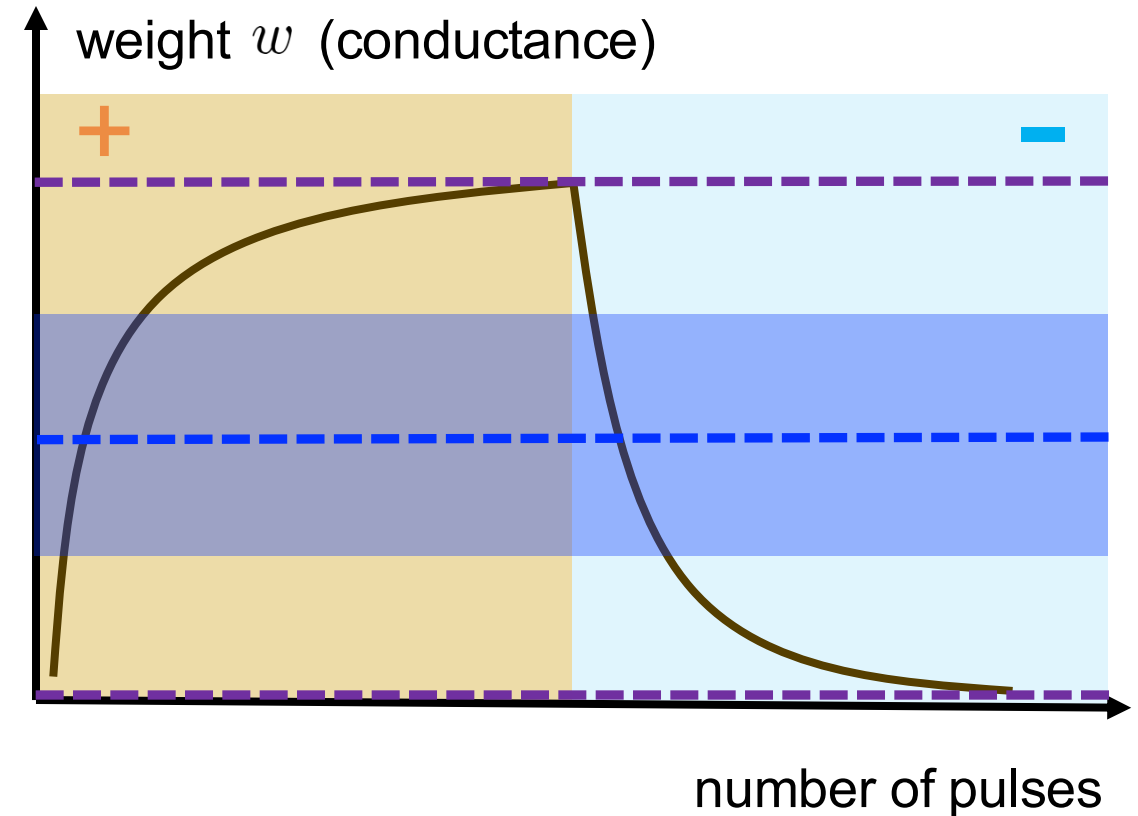
$$\frac{1}{K} \sum_{k=0}^{K-1} \mathbb{E}[\|\nabla f(W_k)\|^2] \leq O\left(\sqrt{\frac{\sigma^2}{K}} \frac{1}{1 - 33P_{\max}^2/\tau^2}\right)$$

Analog momentum eliminates the error when $P_{\max} < \tau$!

Tiki-Taka enlarges the stable region of non-ideal devices

Analog SGD uses **narrow** region
(weight stays near symmetric point)

Tiki-Taka uses **wider** dynamic range
(optimal W can be almost everywhere)



Algorithm innovation relaxes the requirement for material!

Governed by complicated physic evolution, the response factor can be **highly non-linear**

$$W_{k+1} = \begin{cases} W_k + \Delta w \cdot q_+(W_k) , & \Delta w \geq 0, \\ W_k + \Delta w \cdot q_-(W_k) , & \Delta w < 0, \end{cases}$$

Systematic analysis for generic RPU device?

Beyond asymmetric linear device

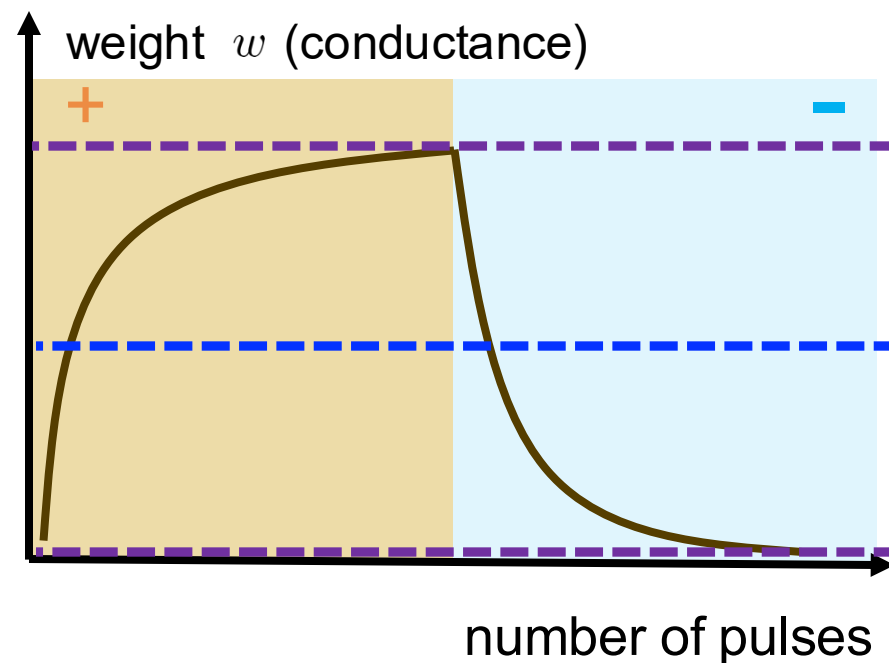
Symmetric attraction / bounded weight properties for asymmetric linear device

$$W_{k+1} = \begin{cases} W_k + \Delta w \cdot q_+(W_k) , & \Delta w \geq 0, \\ W_k + \Delta w \cdot q_-(W_k) , & \Delta w < 0, \end{cases}$$

Symmetric
decomposition

$$F(w) := \frac{1}{2}(q_-(w) + q_+(w))$$
$$G(w) := \frac{1}{2}(q_-(w) - q_+(w))$$

$$W_{k+1} = W_k + \Delta W \odot F(W_k) - |\Delta W| \odot G(W_k)$$



What are the “essential” properties of RPU devices?

Physically reasonable assumptions

Continuity: $q_+(\cdot), q_-(\cdot)$ are continuous function

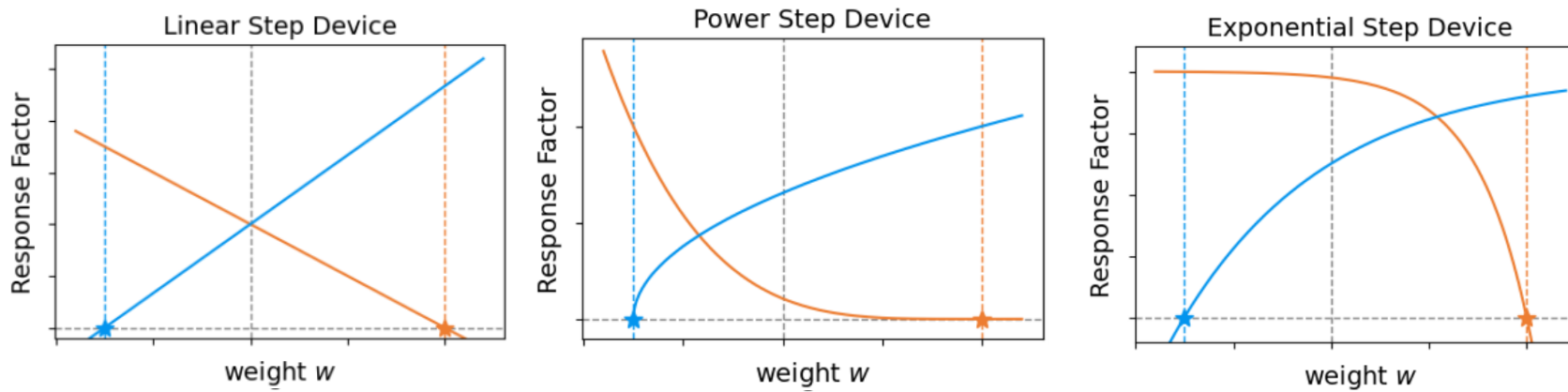
Physic properties should change gradually

Saturation: unique constants $\tau_{\max} \geq \tau_{\min}$ such that $q_+(\tau_{\max}) = q_-(\tau_{\min}) = 0$

Conductance can neither increase nor decrease unlimitedly

Positive-definiteness: $q_+(\cdot) \geq 0, q_-(\cdot) \geq 0$

Positive pulse incurs weight increasement and vice versa



Intrinsic bounded weight

Theorem 4 (Bounded)

If the increment $|\Delta w| \leq \min\{\tau_{\min}, \tau_{\max}\}$, it holds for all element i that

$$\tau_{\min} \leq [W_k]_i \leq \tau_{\max}, \forall k \in \mathbb{N}.$$

Or equivalently,

$$-F([W_k]_i) \leq G([W_k]_i) \leq F([W_k]_i), \forall k \in \mathbb{N}.$$

Natural extension from asymmetric linear device

Rewrite the inequality as $\left\| \frac{G(W)}{F(W)} \right\|_{\infty} \leq 1 \quad \longrightarrow$

Equivalent objective

$$\arg \min_{W \in \mathbb{R}^D} f(W) \quad \text{s.t.} \quad \left\| \frac{G(W)}{F(W)} \right\|_{\infty} \leq 1.$$

Analog training implicitly optimizes a constraint problem

Implicit bias of analog training

$$\arg \min_{W \in \mathbb{R}^D} f(W) \quad \text{s.t.} \quad \left\| \frac{G(W)}{F(W)} \right\|_{\infty} \leq 1.$$

Theorem 4 (KKT point)

A point W^* is a KKT point of the constraint problem iff

$$\frac{G(W^*)}{F(W^*)} \in -\text{sign}(\nabla f(W^*)).$$

Define a natural convergence metric

$$E_K := \frac{1}{K} \sum_{k=0}^{K-1} \|\nabla f(W_k) \odot F(W_k) + |\nabla f(W_k)| \odot G(W_k)\|^2.$$

Evaluate how far from “convergence”

Analog SGD converges to KKT point inexactly

Theorem 5 (Convergence)

Under the above assumption, if the step size is set as $\alpha = O(1/\sqrt{\sigma^2 K})$, K is sufficiently large, Analog SGD satisfies

$$E_K \leq O\left(\sqrt{\frac{\sigma^2}{K}}\right) + \boxed{\sigma^2 S_K^2} \quad \leftarrow \text{asymptotic error}$$

Where $S_K := \frac{1}{K} \sum_{k=0}^{K-1} \left\| \frac{G(W_k)}{\sqrt{F(W_k)}} \right\|_\infty^2$.

Asymptotic error is large if S_k is large (saturation); or σ^2 is large (noisy gradient)

Analog SGD does not converge to KKT point exactly!

Tiki-Taka converges to KKT point exactly

Unique & zero-shifted SP: $q_+(w) = q_-(w)$ iff $w = 0$

Positive pulse incurs weight increasement and vice versa

Theorem 5 (Convergence)

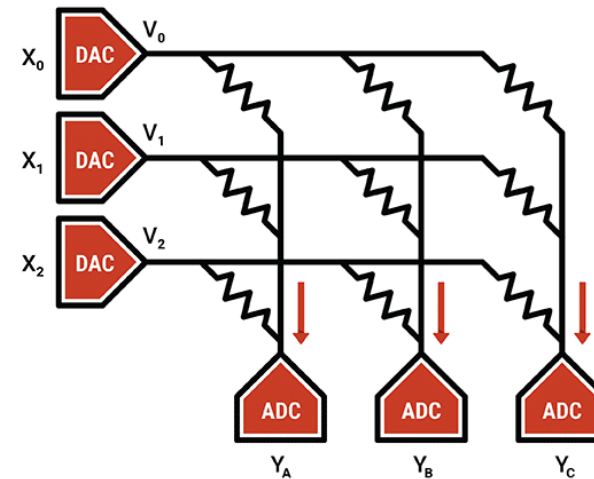
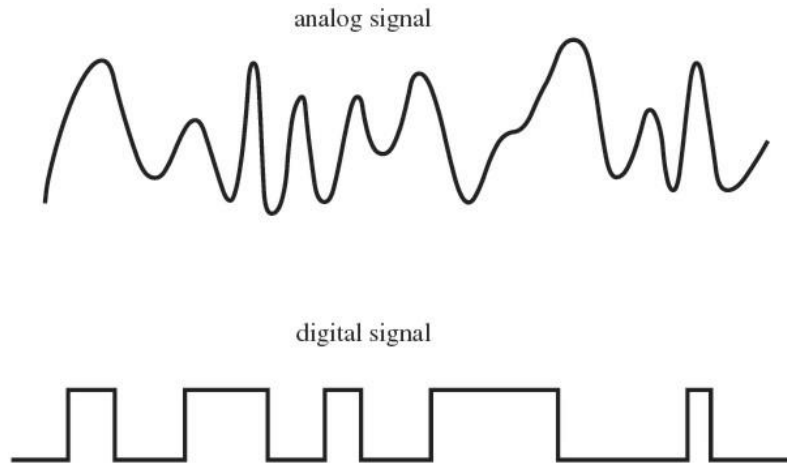
Under the above assumption, if the step size is set as $\alpha = O(1/\sqrt{\sigma^2 K})$, K is sufficiently large, Tiki-Taka satisfies

$$E_K \leq O\left(\sqrt{\frac{\sigma^2}{K}}\right)$$

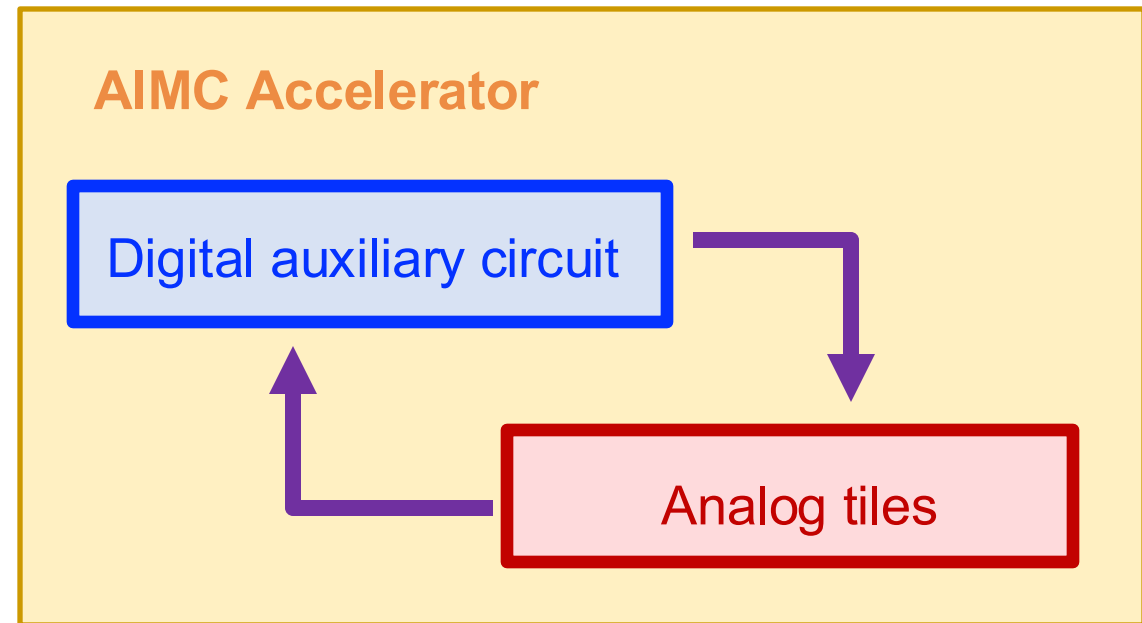
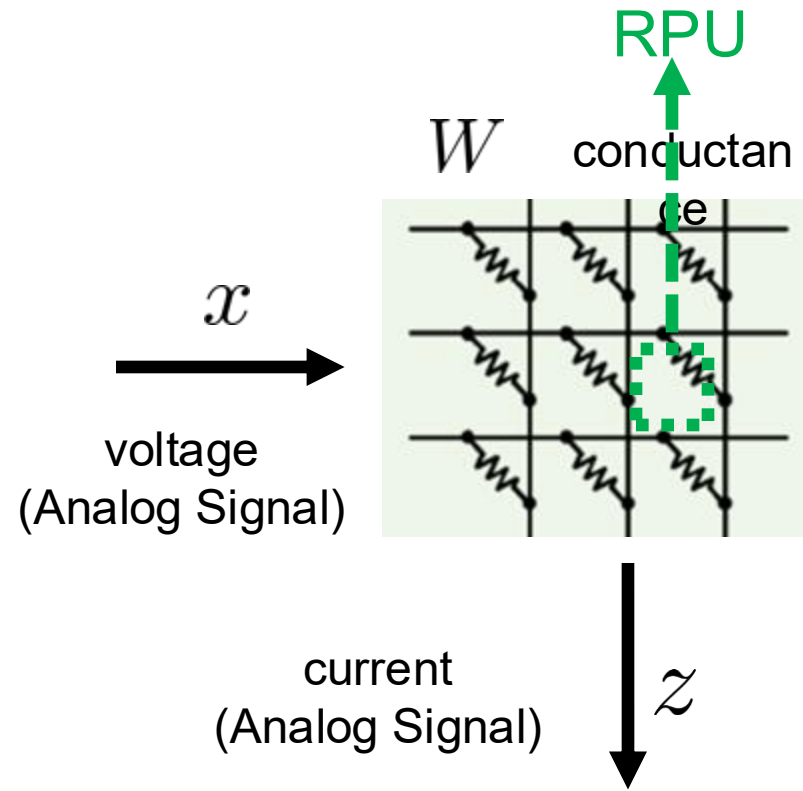
Tiki-Taka (analog momentum) converge to KKT point exactly!

Outline

- Challenges in Analog Training
- From Linear to General Devices
- Advanced Topics in Analog Training

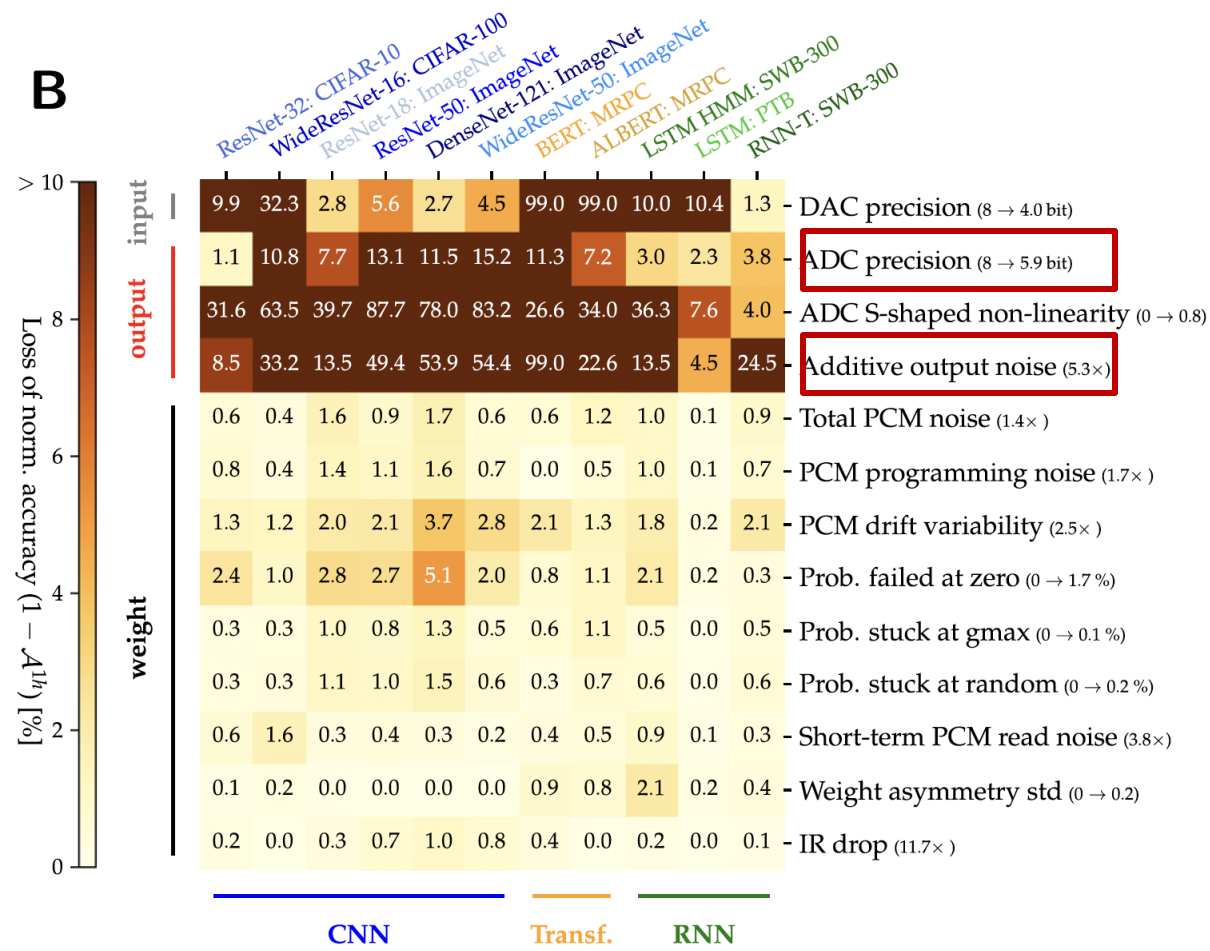


Other non-idealities: reading noise and ADC noise



Analog device has reading noise: additive noise and analog-digital converter error

Other non-idealities: reading noise and ADC noise



Analog device has reading noise: additive noise and analog-digital converter error

TTv2: Algorithm design for “reading noise”

Update operator per pulse

1. Gradient accumulate at analog tile

$$P_{k+1} = \underbrace{U_p \circ U_p \circ \dots \circ U_p}_{\times \text{BL}} (P_k, \text{sign}(\nabla f(W_k; \xi_k)))$$

2. Transfer to digital filter tile

Additive noise can be cancelled out on digital device

$$H_{k+\frac{1}{2}} = (1 - \alpha)H_k + \alpha \left(P_{k+1} + \boxed{\epsilon_k} - \tilde{P}^\diamond \right)$$

3. Update pulse

$$W_{k+1} = U_p \left(W_k, \text{sign} \left(H_{k+\frac{1}{2}} \right) \odot \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

4. Update filter

$$H_{k+1} = H_{k+\frac{1}{2}} \odot \left(1 - \mu_2 \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

TTv2: Algorithm design for “reading noise”

Update operator per pulse

1. Gradient accumulate at analog tile

$$P_{k+1} = \underbrace{U_p \circ U_p \circ \dots \circ U_p}_{\times \text{BL}} (P_k, \text{sign}(\nabla f(W_k; \xi_k)))$$

2. Transfer to digital filter tile

$$H_{k+\frac{1}{2}} = (1 - \alpha)H_k + \alpha \left(P_{k+1} + \epsilon_k - \tilde{P}^\diamond \right)$$

Update when accumulated filter value exceeds threshold

3. Update pulse

$$W_{k+1} = U_p \left(W_k, \text{sign} \left(H_{k+\frac{1}{2}} \right) \odot \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

4. Update filter

$$H_{k+1} = H_{k+\frac{1}{2}} \odot \left(1 - \mu_2 \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

TTv3: Algorithm design for “non-zero shifting”

If the gradient tile P_k is not zero-shifted

1. Auxiliary Chopper variable: $c_k \in \{0,1\}$

$$P_{k+1} = U(P_k, \boxed{c_k} \beta \nabla f(W_k; \xi_k))$$

2. Transfer to digital filter tile

$$H_{k+\frac{1}{2}} = H_k + \alpha \boxed{c_k} (P_{k+1} - \tilde{P}^\diamond)$$

$$c_{k+1} = \begin{cases} c_k, & \text{w.p. } 1 - p_k \\ -c_k, & \text{w.p. } p_k \end{cases}$$

3. Update pulse

$$W_{k+1} = U_p \left(W_k, \text{sign} \left(H_{k+\frac{1}{2}} \right) \odot \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

4. Update filter

$$H_{k+1} = H_{k+\frac{1}{2}} \odot \left(1 - \mu_2 \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

TTv4: Algorithm design for “non-zero shifting”

If the gradient tile P_k is not zero-shifted

1. Auxiliary Chopper variable: $c_k \in \{0,1\}$

$$P_{k+1} = U(P_k, c_k \beta \nabla f(W_k; \xi_k))$$

$$c_{k+1} = \begin{cases} c_k, & \text{w.p. } 1 - p_k \\ -c_k, & \text{w.p. } p_k \end{cases}$$

2. Transfer to digital filter tile

Tracking the symmetric point

$$H_{k+\frac{1}{2}} = H_k + \alpha c_k (P_{k+1} - \tilde{P}_k^\diamond) \quad \tilde{P}_{k+1}^\diamond = (1 - \eta) \tilde{P}_k^\diamond + \eta P_{k+1}$$

3. Update pulse

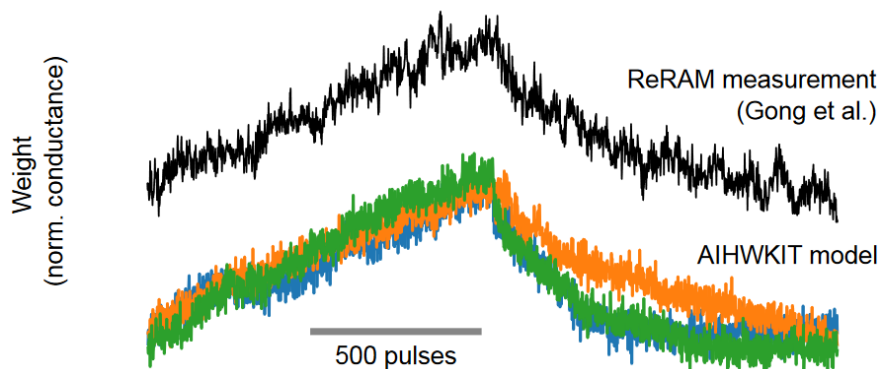
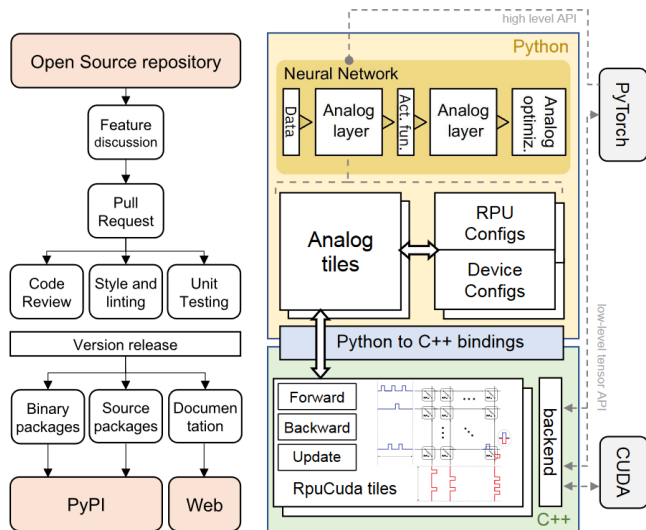
$$W_{k+1} = U_p \left(W_k, \text{sign} \left(H_{k+\frac{1}{2}} \right) \odot \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

4. Update filter

$$H_{k+1} = H_{k+\frac{1}{2}} \odot \left(1 - \mu_2 \mathbb{I} \left\{ \left| H_{k+\frac{1}{2}} \right| \geq \mu_1 \Delta w_{\min} \right\} \right)$$

AIMC hardware simulator

Open-source IBM Analog Hardware Acceleration Kit (**AIHWKit**) is a good tool to explore analog AI



AIHWKit provides analog device simulations across different granular levels

<https://github.com/IBM/aihwkit>

<https://aihwkit.readthedocs.io/en/latest/index.html>

AIMC hardware simulator (con't)

AIHWKit is a Pytorch-based AIMC hardware simulator

Search docs

GET STARTED

- Installation
- Advanced installation guide
- Using the PyTorch integration
- Glossary

ANALOG AI CONCEPTS

- Analogue AI
- What is analogue AI and an analogue chip?
- Analogue AI Hardware
- Advantages and Challenges

CLOUD/COMPOSER

- Analogue AI Cloud Composer Overview
- Composer CLI

USING THE SIMULATOR

- Using aiHWKit Simulator

ANALOG DNN TRAINING

- Specialized Update Algorithms
- Analogue Training Presets

Analog AI

What is analogue AI and an analogue chip?

In a traditional hardware architecture, computation and memory are siloed. Information is moved back and forth between computation and memory operation is performed, creating a limitation called the von Neumann bottleneck.

In-memory computing delivers radical performance improvements by confining memory in a single device, eliminating the von Neumann bottleneck. By leveraging properties of memory devices, computation happens at the same place where data is stored, drastically reducing energy consumption. Many types of memory devices (PCM), resistive random-access memory (RRAM), and Flash memory computing [1]. Because there is no movement of data, tasks can be performed faster and with much less energy. This is different from a conventional architecture where data is transferred from the memory to the CPU every time a computation is performed.

Processing unit and conventional memory

Processing unit and in-memory computing

IBM Analog Hardware Acceleration Kit

pypi v0.9.2 docs passing Build Status license Apache 2.0 arXiv 2104.06714

Description

IBM Analog Hardware Acceleration Kit is an open source Python toolkit for exploring and using the capabilities of in-memory computing devices in the context of artificial intelligence.

⚠ This library is currently in beta and under active development. Please keep an eye for improvements, new features and bug fixes in upcoming releases.

The toolkit consists of two main components:

Pytorch integration

A series of primitives and features that allow using the toolkit within PyTorch.

Files

- master
- tutorial
 - analog_training.ipynb
 - extending_functionality.ipynb
 - hw_aware_training.ipynb
 - post_training_input_range_calibration.ipynb
 - LeNet5.ipynb
 - README.md
 - analog_device_nonidealities_tutorial.ipynb
 - analog_fusion.ipynb
 - analog_sensitivity_LeNet5.ipynb
 - analog_training_LeNet5.ipynb
 - analog_training_LeNet5_TT.ipynb

aihwkit / notebooks / LeNet5.ipynb

Preview Code Blame 1949 lines (1949 loc) · 63.9 KB

IBM Analog Hardware Acceleration Kit (AIHWKIT)

The IBM Analog Hardware Acceleration Kit (AIHWKIT) is an open source Python toolkit for exploring and using the capabilities of in-memory computing devices such as PCM in the context of artificial intelligence. The PyTorch integration consists of a series of primitives and features that allow using the toolkit within PyTorch.

The github repository can be found at: <https://github.com/IBM/aihwkit>

To learn more about Analogue AI and the hardware behind it, refer to this webpage: <https://analogue-ai-demo.mybluemix.net/hardware>

The first thing to do is to install the AIHWKIT and dependencies in your environment. The preferred way to install this package is by using the Python package index:

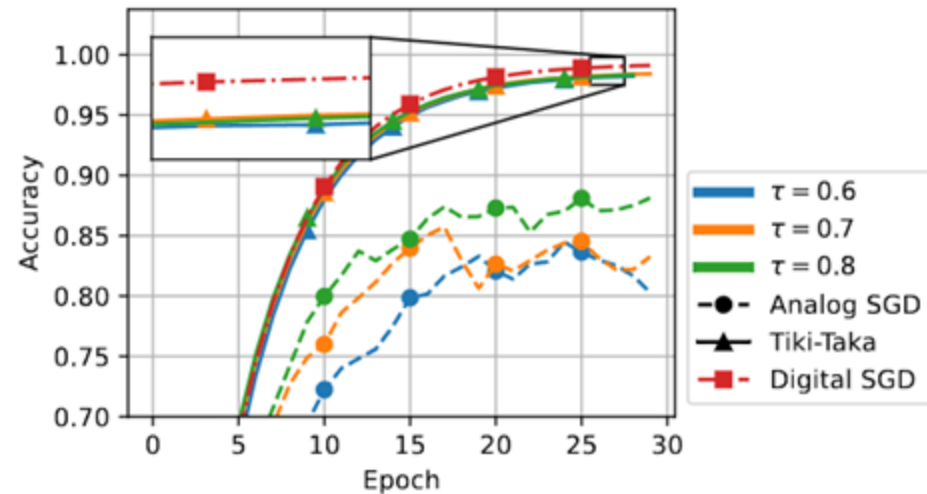
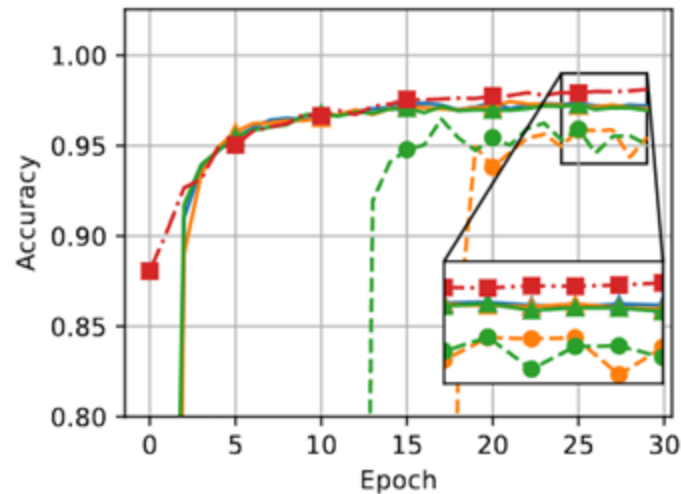
```
In [ ]: # To install the cpu-only enabled kit, un-comment the line below
# pip install aihwkit

# To install the GPU-enabled wheel go to https://aihwkit.readthedocs.io/en/latest/advanced_install.html#install-gpu-wheel
# and copy the option on GPU options that best suits your environment and paste it below and run the cell. For example:
# !wget https://aihwkit-gpu-demo.s3.us-east-1.amazonaws.com/aihwkit-0.9.2+cuda121-cp310-cp310-manylinux2014_x86_64.whl
# !pip install aihwkit-0.9.2+cuda121-cp310-cp310-manylinux2014_x86_64.whl
```

Detailed documents / active community / interactive tutorial

Simulations: Computer vision task

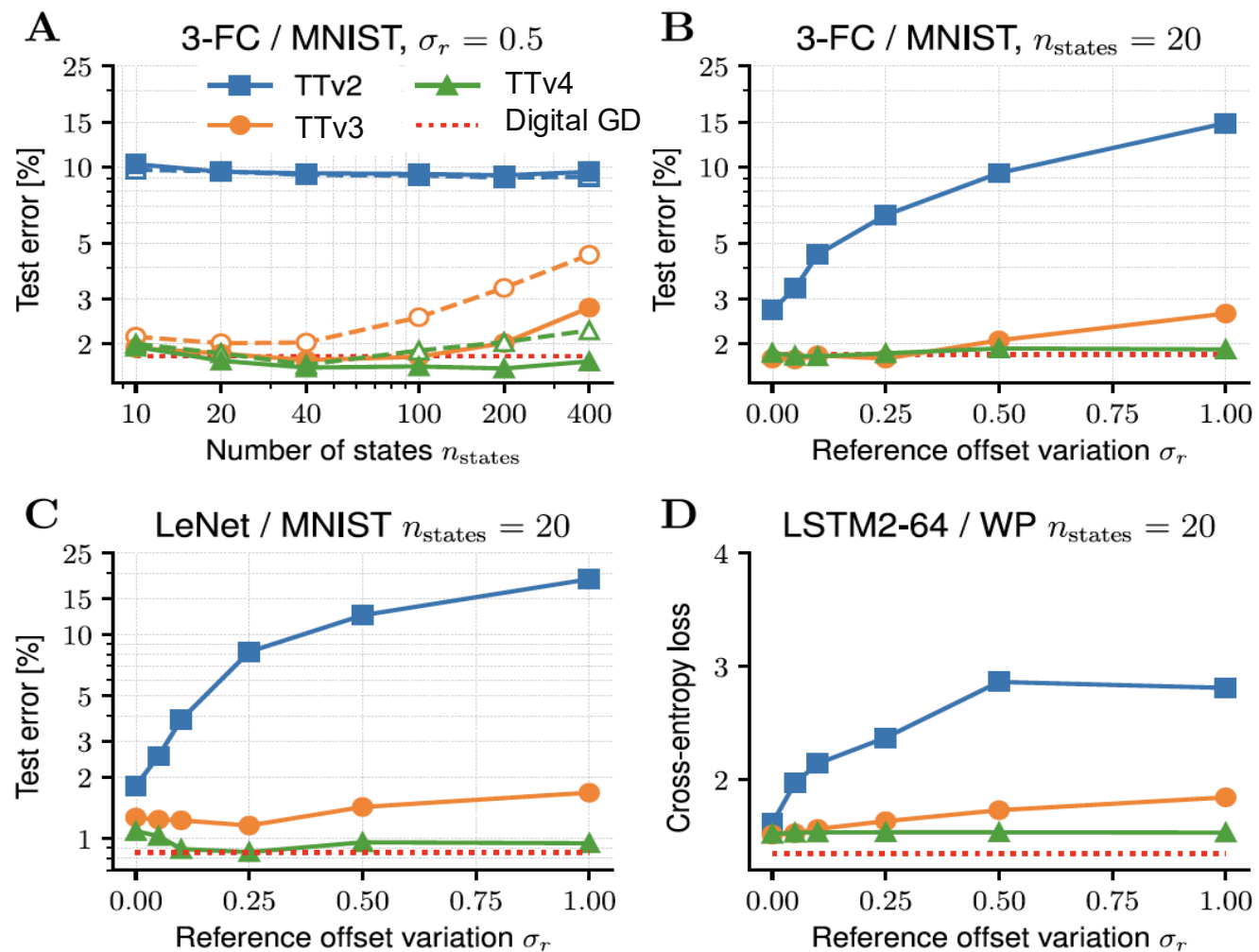
Image Classification: MNIST + fully connect network (left) / convolution network (right)



	$\tau = 0.70$	$\tau = 0.78$	$\tau = 0.80$	$\tau = 0.6$	$\tau = 0.7$	$\tau = 0.8$
D SGD	98.01 ± 0.08			99.24 ± 0.02		
A SGD	10.85 ± 0.54	95.48 ± 0.65	95.32 ± 0.51	78.88 ± 2.57	82.17 ± 4.26	88.45 ± 3.60
TT	97.22 ± 0.08	97.15 ± 0.08	97.08 ± 0.12	98.43 ± 0.13	98.56 ± 0.05	98.49 ± 0.12

Tiki-Taka performs better than Analog SGD

Comparison of different versions of Tiki-Taka



TTv4>TTv3>TTv2

Simulations: Speech recognition task

Fine-tuning **Wav2Vec2** model on **LibriSpeech** dataset

	Word Error Rate
Digital AdamW	4.5%
Analog SGD	6.9%
Tiki Taka	5.2%

Original text:

HE BEGAN A CONFUSED COMPLAINT AGAINST THE WIZARD WHO
HAD VANISHED BEHIND THE CURTAIN ON THE LEFT

Prediction

HE BEGAN A CONFUSED COMPLAINT AGAINST THE WIZARD WHO
HAD **VANISH** BEHIND THE CURTAIN ON THE LEFT

Tiki-Taka performs better than Analog SGD

Summary

Analog device non-ideality affects training...

Math models to capture device-specific performance!

Theory-inspired algorithms address them!

Training on Analog device has implicit bias

Tiki-Taka mitigates the asymptotic error of Analog SGD

TTv2/v3/v4 solves the reading noise and non-zero shift issues

